Retail Clickstream Analytics

Ingestion Layer

1. Understand the source dataset

Your source is the E-commerce behavior data from a multi-category store.

Its records describe customer interactions with products. The main fields include:

Field	Meaning
`event_time`	Time of the recorded interaction
`event_type`	Interaction type, such as view, cart, or purchase
`product_id`	Product identifier
`category_id`	Category identifier
`category_code`	Category hierarchy, when available
`brand`	Product brand, when available
`price`	Price recorded for the event
`user_id`	Pseudonymous user identifier
`user_session`	Browsing session identifier

> we'll use it to construct a consistent synthetic operational dataset.

What should we generate?

1. `customers` — customer records mapped from source user IDs.
2. `products` — product records derived from observed product IDs.
3. `orders` — synthetic order headers derived from purchase events.
4. `order_items` — synthetic order lines derived from purchase events.
5. `inventory` — synthetic current stock levels.

2. Define how each table will be generated

The dataset contains multiple events for the same customer and product. It also explicitly notes that multiple purchase events within one session can belong to the same order. Therefore, we should not create a new order for every purchase event.

Use the following rules.

OLTP table	Source	Generation rule
`customers`	`user_id`	One record per distinct source user
`products`	`product_id`	One record per distinct product, with its observed attributes
`orders`	Purchase events	One synthetic order per eligible customer session

OLTP table	Source	Generation rule
`order_items`	Purchase events	One line per product purchased in that session, aggregating repeated product entries according to a documented deduplication policy
`inventory`	No direct source	Generate synthetic stock quantities for each product

For the initial implementation, use these assumptions:

1. Each source `user_id` maps to one synthetic customer.
2. Each source `product_id` maps to one product.
3. Each eligible session with purchase events maps to one synthetic order.
4. Each purchase event represents one unit **only as a modeling assumption**, because the source does not provide a purchase quantity.
5. Order totals are calculated from the prices recorded in the purchase events.
6. Payment status, shipping information, and inventory quantities are synthetic—not historical facts.

A key modeling decision: What should happen to sessions without purchases?

They should not produce order records.

However, their views, cart additions, and other events remain in your clickstream dataset. This is important because you need sessions with no purchases to measure conversion rates and cart abandonment.

3. Generate the datasets using Python

Step 1 — Prepare the Python environment

Inside your project:

```
mkdir -p data/processed
mkdir -p scripts
python3 -m venv .venv
source .venv/bin/activate
pip install pandas numpy faker kafka-python "dbt-core==1.12.6" "dbt-snowflake==1.12.4"
```

Place your selected Kaggle CSV in:

```
data/
└── 2019-Oct.csv
```

October 2019 is a reasonable starting point, but use the file you have downloaded.

Step2 — Explore the Complete CSV Dataset

```
python3 explore_stream_data.py
```

```
(.venv) mohamed-elgohary@LOQ-U:~/WorkSpace/retail-clickstream-analytics$ python3 explore_data.py
============================================================
DATASET INFORMATION
```

```
============================================================
File: /home/mohamed-elgohary/WorkSpace/retail-clickstream-analytics/data/2019-Oct.csv
File size: 5.28 GB

Loading dataset...
Dataset loaded successfully.

============================================================
DATASET DIMENSIONS
============================================================
Total rows: 42,448,764
Total columns: 9

Columns:
['event_time', 'event_type', 'product_id', 'category_id', 'category_code', 'brand', 'price', 'user_id', 'user_sessio
n']

Data types:
event_time       datetime64[us, UTC]
event_type                       str
product_id                     int64
category_id                    int64
category_code                    str
brand                            str
price                          float64
user_id                        int64
user_session                     str
dtype: object

============================================================
MISSING VALUES
============================================================
               missing_count  missing_percentage
category_code       13515609               31.84
brand                6117080               14.41
event_time                 0                0.00
product_id                 0                0.00
event_type                 0                0.00
category_id                0                0.00
price                      0                0.00
user_id                    0                0.00
user_session               2                0.00

============================================================
EVENT DISTRIBUTION
============================================================
            event_count  percentage
event_type
view           40779399       96.07
cart             926516        2.18
purchase         742849        1.75

============================================================
UNIQUE ENTITIES
============================================================
Unique users: 3,022,290
Unique sessions: 9,244,421
Unique products: 166,794
Unique categories: 624

============================================================
SESSION STATISTICS
============================================================
Total sessions: 9,244,421
Average events per session: 4.59
Median events per session: 2
Maximum events in a session: 1,159
```

```
Events per session distribution:
count    9.244421e+06
mean     4.591825e+00
std      6.766295e+00
min      1.000000e+00
25%      1.000000e+00
50%      2.000000e+00
75%      5.000000e+00
max      1.159000e+03

============================================================
PURCHASE ANALYSIS
============================================================
Total purchase events: 742,849
Users with purchase events: 347,118
Sessions with purchase events: 629,560
Unique products purchased: 42,241

Purchase events per purchasing session:
count    629560.000000
mean          1.179949
std           0.600796
min           1.000000
25%           1.000000
50%           1.000000
75%           1.000000
max          48.000000

Sessions with multiple purchase events: 80,261

============================================================
EVENT TIME RANGE
============================================================
Earliest event: 2019-10-01 00:00:00+00:00
Latest event: 2019-10-31 23:59:59+00:00

Exploration completed.
```

Step3 — Initial run — build all five tables

Pass both monthly files together:

```
python generate_oltp.py \
  --input data/2019-Oct.csv data/2019-Nov.csv \
  --mode replace
```

This creates or rebuilds:

```
data/processed/
├── customers.csv
├── products.csv
├── orders.csv
├── order_items.csv
├── inventory.csv
└── oltp_build.sqlite
```

The SQLite database is an intermediate work database. The five CSV files are the outputs intended for your pipeline.

```
(.venv) mohamed-elgohary@LOQ-U:~/WorkSpace/retail-clickstream-analytics$ python generate_oltp.py \
  --input data/2019-Oct.csv  \
  --mode replace
Mode: replace
Input files: 1
Chunk size: 200,000
SQLite work database: data/processed/oltp_build.sqlite

Processing /home/mohamed-elgohary/WorkSpace/retail-clickstream-analytics/data/2019-Oct.csv...
  rows read: 42,448,764 | purchase events: 742,849

Total valid rows processed: 42,448,764
Purchase events processed: 742,849

Merging customers...
  new customers generated: 3,000,000
  new customers generated: 3,022,290
Merging products...
  new products inserted: 150,000
  new products inserted: 166,794
Merging orders and order items...
Merging inventory...

Validation
  customers with missing email: 0
  duplicate customer emails: 0
  products with invalid price: 0
  order items with non-positive quantity: 0
  order items missing order: 0
  order items missing product: 0
  inventory rows missing product: 0
  order totals differing from line totals: 0
  products missing inventory: 0
  All validation checks passed.
Writing data/processed/customers.csv...
  3,022,290 rows
Writing data/processed/products.csv...
  166,794 rows
Writing data/processed/orders.csv...
  629,560 rows
Writing data/processed/order_items.csv...
  690,372 rows
Writing data/processed/inventory.csv...
  166,794 rows

Done.
```

Step4 — Add or update data later

Use `upsert` when updating existing outputs:

```
python generate_oltp.py \
  --input data/2019-Oct.csv data/2019-Nov.csv data/2019-Dec.csv \
  --mode upsert
```

For correct cross-month aggregation, **include all monthly source files processed so far** when running in `upsert` mode. The script merges records using stable identifiers, preserves existing synthetic customer attributes and inventory values, and updates order items and totals from the combined source data.

Setup OLTP Database

Step 1 —Pull the PostgreSQL image

We'll use the official PostgreSQL image with **PostgreSQL 17**.

Run:

```
docker pull postgres:17
```

This downloads the image without starting a container.

Step 2 —Start PostgreSQL using Docker

run:

```
docker run -d \
  --name retail-postgres \
  -e POSTGRES_USER=retail_admin \
  -e POSTGRES_PASSWORD='admin1234' \
  -e POSTGRES_DB=retail_oltp \
  -p 5050:5432 \
  -v retail_pgdata:/var/lib/postgresql/data \
  postgres:17
```

Setting	Purpose
`retail-postgres`	Container name
`retail_admin`	PostgreSQL user
`retail_oltp`	Database for your OLTP tables
`5432:5432`	Exposes PostgreSQL on port 5432 of your computer
`retail_pgdata`	Persistent Docker volume for database files
`postgres:17`	PostgreSQL 17 image

Verify that PostgreSQL is ready

Run:

```
docker exec -it retail-postgres \
  pg_isready -U retail_admin -d retail_oltp
```

Expected output:

```
/var/run/postgresql:5432 - accepting connections
```

```
# Open a Bash shell inside the container

docker exec -it retail-postgres bash

# Then connect to PostgreSQL:

psql -U retail_admin -d retail_oltp
```

Step 3 — Create the tables

At the `retail_oltp=#` prompt, execute the following SQL.

```
CREATE TABLE customers (
    customer_id BIGINT PRIMARY KEY,
    first_name VARCHAR(100),
    last_name VARCHAR(100),
    email VARCHAR(255) UNIQUE,
    country VARCHAR(100),
    registration_date TIMESTAMP
);

CREATE TABLE products (
    product_id BIGINT PRIMARY KEY,
    category_id BIGINT,
    category_code TEXT,
    brand VARCHAR(255),
    price NUMERIC(12, 2),
    cost NUMERIC(12, 2)
);

CREATE TABLE orders (
    order_id TEXT PRIMARY KEY,
    customer_id BIGINT REFERENCES customers(customer_id),
    order_timestamp TIMESTAMP,
    order_total NUMERIC(14, 2),
    order_status VARCHAR(50)
);

CREATE TABLE order_items (
    order_item_id BIGINT PRIMARY KEY,
    order_id TEXT REFERENCES orders(order_id),
    product_id BIGINT REFERENCES products(product_id),
    quantity INTEGER,
    unit_price NUMERIC(12, 2),
    line_total NUMERIC(14, 2),
    first_purchase_time TIMESTAMP,
    UNIQUE (order_id, product_id)
);

CREATE TABLE inventory (
    product_id BIGINT PRIMARY KEY REFERENCES products(product_id),
    stock_quantity INTEGER,
    reorder_level INTEGER,
    inventory_origin VARCHAR(100)
);
```

Step 4 —Create the destination directory inside the container

Run this from your Ubuntu terminal:

```
docker exec retail-postgres mkdir -p /tmp/retail-processed
```

The syntax is:

```
docker cp <source> <destination>
```

This command **copies a file between your Ubuntu host and a Docker container**.

Step 5 —Copy the files

```
docker cp customers.csv retail-postgres:/tmp/retail-processed/
docker cp products.csv retail-postgres:/tmp/retail-processed/
docker cp orders.csv retail-postgres:/tmp/retail-processed/
docker cp order_items.csv retail-postgres:/tmp/retail-processed/
docker cp inventory.csv retail-postgres:/tmp/retail-processed/
```

Step6 — Load CSV to retail_oltp Database

Open PostgreSQL:

```
docker exec -it retail-postgres psql -U retail_admin -d retail_oltp
```

Run these commands one by one.

```
\copy customers FROM '/tmp/retail-processed/customers.csv' WITH (FORMAT CSV, HEADER TRUE);
```

```
\copy products FROM '/tmp/retail-processed/products.csv' WITH (FORMAT CSV, HEADER TRUE);
```

```
\copy orders FROM '/tmp/retail-processed/orders.csv' WITH (FORMAT CSV, HEADER TRUE);
```

```
\copy order_items FROM '/tmp/retail-processed/order_items.csv' WITH (FORMAT CSV, HEADER TRUE);
```

```
\copy inventory FROM '/tmp/retail-processed/inventory.csv' WITH (FORMAT CSV, HEADER TRUE);
```

Important: Run each command separately and wait for its result before executing the next. If a table was already imported successfully, don't import it again.

Real-Time Clickstream Streaming Pipeline

Overview

We simulate user activity by reading historical clickstream events and publishing them to Kafka at a controlled rate. Kafka acts as the event-streaming layer, and Kafka Connect delivers the events to Snowflake for downstream analytics.

Architecture:

`Python Producer → Apache Kafka → Kafka Connect + Snowflake Connector → Snowflake RAW`

The pipeline uses Docker Compose to run Kafka and Kafka Connect. Snowflake key-pair authentication secures the connection between Kafka Connect and Snowflake.

Prerequisites

Before starting, ensure you have:

- Docker Engine or Docker Desktop with Docker Compose.
- Python and the project's virtual environment.
- The project repository cloned locally.
- Access to the Snowflake account and permission to configure the service user.
- The required project files, including the Dockerfile, Compose file, connector configuration, and producer script.

Run the following commands from the project root:

```
cd ~/WorkSpace/retail-clickstream-analytics
```

Step 1 — Build the Kafka Connect image

Build the custom image containing the Snowflake Kafka Connector:

```
docker build \
  -t retail-snowflake-connect:local \
  -f streaming/connect/Dockerfile \
  streaming/connect
```

The Dockerfile uses the Confluent Kafka Connect base image and installs the Snowflake connector plugin.

This step builds the image; it does not start the containers.

Step 2 — Configure Snowflake RSA Key-Pair Authentication

The Snowflake Kafka Sink Connector uses RSA key-pair authentication instead of a password. A 2048-bit RSA private key is kept on the Ubuntu host, while its matching public key is registered with the Snowflake service user `RETAIL_KAFKA_CONNECT_USER`.

> **Security:** Never commit `rsa_key.p8` or `connector-secrets.properties` to GitHub, print the private key, or share the secrets file. These files contain unencrypted private-key material.

2.1 Create a Secure Directory

Run on the Ubuntu host:

```
mkdir -p ~/.config/retail-clickstream/snowflake
chmod 700 ~/.config/retail-clickstream/snowflake
```

2.2 Generate and Validate the RSA Key Pair

Generate a 2048-bit RSA private key in unencrypted PKCS#8 format:

```
openssl genpkey \
  -algorithm RSA \
  -out ~/.config/retail-clickstream/snowflake/rsa_key.p8 \
  -pkeyopt rsa_keygen_bits:2048
```

The original streaming README also provides this alternative command for the same key-generation step (use **one** generation method, not both):

```
openssl genrsa 2048 | openssl pkcs8 \
  -topk8 \
  -inform PEM \
  -outform PEM \
  -nocrypt \
  -out ~/.config/retail-clickstream/snowflake/rsa_key.p8
```

Generate the corresponding public key:

```
openssl pkey \
  -in ~/.config/retail-clickstream/snowflake/rsa_key.p8 \
  -pubout \
  -out ~/.config/retail-clickstream/snowflake/rsa_key.pub
```

Set file permissions and validate the private key:

```
chmod 600 ~/.config/retail-clickstream/snowflake/rsa_key.p8
chmod 644 ~/.config/retail-clickstream/snowflake/rsa_key.pub

openssl pkey \
  -in ~/.config/retail-clickstream/snowflake/rsa_key.p8 \
  -check -noout
```

Expected result: `Key is valid`.

```
~/.config/retail-clickstream/snowflake/
├── rsa_key.p8
└── rsa_key.pub
```

The private key stays on the host. The public key is registered in Snowflake.

2.3 Register the Public Key in Snowflake

Extract the public key body without PEM header or footer:

```
sed '/-----BEGIN PUBLIC KEY-----/d; /-----END PUBLIC KEY-----/d' \
  ~/.config/retail-clickstream/snowflake/rsa_key.pub
```

Copy the output and run the following SQL using a role authorized to alter the Snowflake user:

```
ALTER USER RETAIL_KAFKA_CONNECT_USER
SET RSA_PUBLIC_KEY = 'PASTE_PUBLIC_KEY_BODY_HERE';
```

Replace the placeholder with the actual public-key body, excluding the `BEGIN PUBLIC KEY` / `END PUBLIC KEY` lines. The user must match the service user in the Kafka connector configuration.

Verify public-key registration:

```
DESC USER RETAIL_KAFKA_CONNECT_USER;
```

Check the `RSA_PUBLIC_KEY_FP` property.

Reference: Snowflake Key-Pair Authentication.

2.4 Create the Connector Secrets File

Kafka Connect reads the private key from a properties file mounted at `/secrets/snowflake/connector-secrets.properties` inside its container. Create the host-side file:

```
KEY_DIR="$HOME/.config/retail-clickstream/snowflake"

{
  printf 'snowflake.private.key='
  sed '/-----BEGIN PRIVATE KEY-----/d; /-----END PRIVATE KEY-----/d' \
    "$KEY_DIR/rsa_key.p8" | tr -d '\n'
  printf '\n'
} > "$KEY_DIR/connector-secrets.properties"

chmod 600 "$KEY_DIR/connector-secrets.properties"
```

The file has a single property (placeholder shown; **never** publish the real value):

```
snowflake.private.key=<PRIVATE_KEY_CONTENT>
```

Verify only that the file exists, without printing its contents:

```
test -f "$HOME/.config/retail-clickstream/snowflake/connector-secrets.properties" \
  && echo "Secrets file exists"
```

2.5 Configure the Docker Compose Secrets Mount

In `streaming/kafka/compose.yaml`, add the following **read-only mount** under the existing `connect` service's `volumes` section. Replace the example username with the Ubuntu host username:

```
volumes:
  - /home/YOUR_LINUX_USERNAME/.config/retail-clickstream/snowflake:/secrets/snow
flake:ro
```

The `connect` environment also needs the Kafka Connect file configuration provider:

```
CONNECT_CONFIG_PROVIDERS: file
CONNECT_CONFIG_PROVIDERS_FILE_CLASS: org.apache.kafka.common.config.provider.Fil
eConfigProvider
```

Do not create a second `volumes` key; merge the mount into the existing service configuration. The host directory is mounted read-only into the container.

2.6 Configure the Snowflake Connector Authentication Properties

In `streaming/config/snowflake-sink.json`, confirm that the existing connector `config` object includes:

```
{
  "snowflake.url.name": "https://YOUR_ACCOUNT_IDENTIFIER.snowflakecomputing.co
m",
  "snowflake.user.name": "RETAIL_KAFKA_CONNECT_USER",
  "snowflake.role.name": "RETAIL_KAFKA_CONNECT_ROLE",
  "snowflake.authenticator": "snowflake_jwt",
  "snowflake.database.name": "RETAIL_CLICKSTREAM_DB",
  "snowflake.schema.name": "RAW",
  "snowflake.private.key": "${file:/secrets/snowflake/connector-secrets.properti
es:snowflake.private.key}"
}
```

> This is a **configuration excerpt**, not the complete connector JSON. Preserve the existing `connector.class`, `topics`, converters, and topic-to-table mapping. Replace the Snowflake account URL placeholder with the actual account URL. This assumes the service user and required grants already exist.

Reference: Snowflake Kafka Connector Setup.

2.7 Validate the Configuration

Check the connector JSON syntax:

```
python -m json.tool streaming/config/snowflake-sink.json
```

When the Kafka Connect service is running, any changed Compose mounts or environment settings must be applied through a suitable Compose recreate/start, and updated connector configuration must be submitted through the REST API (see Steps 3 and 5). For a simple service restart after the configuration is in place, the RSA guide provides:

```
docker compose -f streaming/kafka/compose.yaml restart connect
```

Review connector logs without exposing credentials:

```
docker logs --tail 100 retail-connect
```

Step 3 — Start Kafka and Kafka Connect

Build any configured custom images and start the services:

```
docker compose \
  -f streaming/kafka/compose.yaml \
  up -d --build
```

The main services are:

Container	Purpose	Local port
`retail-kafka`	Receives clickstream events	`9094`
`retail-connect`	Transfers Kafka events to Snowflake	`8083`

Kafka Connect's REST API is available at `http://localhost:8083`.

Step 4 — Create the Kafka topic

Topic auto-creation is disabled, so create the topic explicitly before running the producer:

```
docker exec retail-kafka \
  /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --create \
  --topic clickstream_events \
  --partitions 1 \
  --replication-factor 1
```

Verify that the topic exists:

```
docker exec retail-kafka \
  /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --list
```

Expected topic:

```
clickstream_events
```

If the topic already exists, verify it rather than attempting to recreate it.

Step 5 — Register or Update the Snowflake Sink Connector

The connector named `RETAIL_CLICKSTREAM_SNOWFLAKE_SINK` maps Kafka events to Snowflake using the topic, account, role, database, schema, and private-key configuration from `streaming/config/snowflake-sink.json`.

Check whether the connector already exists:

```
curl -sS http://localhost:8083/connectors
```

5.1 Create the Connector (If It Does Not Exist)

```
curl -sS -X POST \
  -H "Content-Type: application/json" \
  --data @streaming/config/snowflake-sink.json \
  http://localhost:8083/connectors
```

Do not submit a duplicate creation request if the connector has already been registered.

5.2 Update the Connector (If It Already Exists)

If `jq` is available, extract the nested configuration object and update it through the Kafka Connect API:

```
jq '.config' streaming/config/snowflake-sink.json > /tmp/snowflake-sink-config.j
son

curl -sS -X PUT \
  -H "Content-Type: application/json" \
  --data @/tmp/snowflake-sink-config.json \
  http://localhost:8083/connectors/RETAIL_CLICKSTREAM_SNOWFLAKE_SINK/config
```

Keep the rest of the connector configuration intact; authentication is only one part of the complete sink settings.

Step 6 — Verify the Snowflake Connector

Check the connector and task status:

```
curl -sS \
  http://localhost:8083/connectors/RETAIL_CLICKSTREAM_SNOWFLAKE_SINK/status \
  | python3 -m json.tool
```

Both the connector and its task should report:

```
"state": "RUNNING"
```

A `RUNNING` status indicates the connector processes are active, **not** that events have necessarily arrived in Snowflake. Verify actual ingestion against `RETAIL_CLICKSTREAM_DB.RAW.RAW_CLICKSTREAM_EVENTS` in Step 8.

If authentication or connector execution fails, verify the public key registered to `RETAIL_KAFKA_CONNECT_USER`, the matching PKCS#8 private key, the property-file path and property name, the read-only mount, `snowflake_jwt`, and the service role's access to the target objects.

Step 7 — Run or resume the producer

Activate the project's Python virtual environment:

```
cd ~/WorkSpace/retail-clickstream-analytics
source .venv/bin/activate
```

Run the producer:

```
python3 streaming/producer/clickstream_producer.py \
  --limit 1000 \
  --rate 100
```

- `-limit 1000`: Publishes up to 1,000 additional source rows during this run.
- `-rate 100`: Requests a target publishing rate of approximately 100 events per second. Actual throughput can be lower.

Checkpoint behavior

The checkpoint is stored at:

```
streaming/producer/checkpoint.json
```

The producer uses it to resume from the saved source position.

Do not use `--reset-checkpoint` during a normal restart.

The checkpoint tracks events acknowledged by Kafka, not events successfully ingested into Snowflake. Events already in Kafka may still be processed after the producer stops or Kafka Connect restarts.

Resetting the producer checkpoint does not clear Kafka or remove existing Snowflake rows.

Step 8 — Verify the events in Snowflake

After publishing events, query the target table:

```
SELECT
    COUNT(*) AS TOTAL_EVENTS,
    MIN(EVENT_TIME) AS FIRST_EVENT_TIME,
    MAX(EVENT_TIME) AS LAST_EVENT_TIME
FROM RETAIL_CLICKSTREAM_DB.RAW.RAW_CLICKSTREAM_EVENTS;
```

Inspect a sample of recent arrivals:

```sql
SELECT *
FROM RETAIL_CLICKSTREAM_DB.RAW.RAW_CLICKSTREAM_EVENTS
ORDER BY SIMULATED_AT DESC
LIMIT 20;
```

Confirm that the event count increases and that the records contain the expected fields.

Troubleshooting

Kafka Connect is not running

```
docker logs --tail 100 retail-connect
```

The connector is not registered

Register it using the command in Step 5.

The connector is registered but its task has failed

Check the connector status and logs before restarting it. Fix the reported configuration, authentication, or permission issue first.

Snowflake authentication fails

Check that:

- The public key was registered against `RETAIL_KAFKA_CONNECT_USER`.
- The private key corresponds to that public key.
- The secrets file exists at the mounted container path.
- The connector configuration references the correct property.
- The private key file has not been modified or exposed.

Events appear in Kafka but not in Snowflake

Check the Kafka Connect task status and logs, then verify the Snowflake user permissions and target table configuration.

Stop the services

To stop the services without removing their containers or persistent volumes:

```
docker compose -f streaming/kafka/compose.yaml stop
```

To start them again:

```
docker compose -f streaming/kafka/compose.yaml up -d
```

Persistent Kafka data and the producer checkpoint allow the pipeline to resume, subject to the configured retention and checkpoint behavior.

Configuration References

- Snowflake Key-Pair Authentication
- Snowflake Kafka Connector Setup

> **Documentation scope:** This section combines the project's streaming pipeline README and RSA authentication guide. Commands and expected outputs describe the documented workflow; they are not a new execution log.

Silver Layer Implementation — dbt and Snowflake

Project: Retail Clickstream Analytics Pipeline

Role: Data Engineering — Data Preparation, Standardization, Quality Assurance & Technical Monitoring

Technology Stack: Python 3.12, VS Code, PowerShell, dbt Core 1.12.6, dbt- snowflake 1.12.4, Snowflake SQL, Git & GitHub

Database: RETAIL_CLICKSTREAM_DB

Source Schema: RAW

Target Schema: STAGING

Snowflake Warehouse: RETAIL_WH

Role: DBT_SILVER_ROLE

1. Data Profiling & Source Understanding

Technology: Snowflake SQL, Snowsight Workspaces

Type: Read-Only Source Analysis

Status: Completed

1.1 Objective

The objective of the Data Profiling & Source Understanding section was to understand the structure, volume, completeness, uniqueness, consistency, relationships, and streaming characteristics of the source datasets before implementing any cleaning or transformations.

Data profiling was performed using read-only SQL queries executed directly in Snowflake.

The analysis covered five OLTP datasets and one clickstream event dataset.

The purpose was to identify actual data quality issues, establish technical validation requirements, and avoid unnecessary deletion or modification of legitimate source records.

1.2 Profiling Methodology

The profiling process was organized into seven steps.

Step	Analysis	Purpose
1	Source Inventory & Metadata	Inspect available tables, columns, data types, and row counts.
2	NULL & Blank Analysis	Identify missing required or optional values.
3	Uniqueness & Primary Keys	Identify duplicate keys and repeated event payloads.
4	Values, Ranges & Financial Consistency	Detect invalid quantities, prices, stock values, and calculation mismatches.
5	Referential Integrity	Validate relationships between customers, products, orders, order items, inventory, and events.
6	Streaming Behavior & Timestamps	Inspect event types, session identifiers, historical timestamps, and replay behavior.
7	Final Profiling Findings	Translate observations into cleaning, transformation, and validation requirements.

All profiling queries were preserved for documentation and troubleshooting.

1.3 Source Inventory and Data Volume

The initial source inventory contained the following datasets:

Dataset	Initial Verified Rows
RAW_CUSTOMERS	3,022,290
RAW_PRODUCTS	166,794
RAW_ORDERS	629,560
RAW_ORDER_ITEMS	690,372
RAW_INVENTORY	166,794
RAW_CLICKSTREAM_EVENTS	6,000

The five OLTP tables contained approximately 4.68 million records.

The clickstream dataset was subsequently loaded with additional events and a source-level event identifier.

Latest verified clickstream snapshot:

Metric	Result
RAW Clickstream Rows	150,000
STAGING Clickstream Rows	150,000
NULL EVENT_ID Values	0
Distinct EVENT_ID Values	150,000
Duplicate EVENT_ID Rows	0

The clickstream row count is a point-in-time observation and may increase as new streaming batches are ingested.

1.4 Completeness and Missing Values

The initial profiling checks showed strong completeness across the core OLTP datasets.

No missing values were detected in the checked customer, order, order-item, and inventory fields.

Missing descriptive attributes were identified in the product and clickstream datasets.

Initial product findings:

Attribute	Missing Rows
CATEGORY_CODE	93,644
BRAND	43,613

Initial clickstream findings (6,000-event snapshot):

Attribute	Missing Rows
CATEGORY_CODE	1,922
BRAND	792

These attributes provide descriptive information but do not necessarily determine whether a source record is valid.

Decision: Preserve legitimate records with missing optional attributes and introduce standardized display labels and missing-value indicators during staging.

The historical clickstream missing-value counts should not be treated as the current counts for the 150,000-event snapshot without rerunning the relevant profiling queries.

1.5 Uniqueness and Primary Key Analysis

Primary-key checks were performed on the five OLTP datasets.

The initial results showed no duplicate values in the tested primary keys:

- CUSTOMER_ID in RAW_CUSTOMERS.
- PRODUCT_ID in RAW_PRODUCTS.
- ORDER_ID in RAW_ORDERS.
- ORDER_ITEM_ID in RAW_ORDER_ITEMS.
- PRODUCT_ID in RAW_INVENTORY.

The initial clickstream analysis also identified one repeated payload group containing four identical occurrences, corresponding to three additional matching rows.

Payload equality alone does not establish whether events are legitimate repeated user actions or duplicate ingestion attempts.

The current RAW clickstream schema includes an EVENT_ID column of type NUMBER(38,0), supplied by the ingestion pipeline.

The latest verified 150,000-event snapshot contained no NULL or duplicate EVENT_ID values.

Decision: Preserve source EVENT_ID values in staging and validate event-level uniqueness. Payload similarity remains an optional diagnostic check rather than a definitive event-identity rule.

Stable identifiers across retries and replays must still be guaranteed by the ingestion implementation.

1.6 Numerical Validity and Financial Consistency

Numerical profiling checked product prices, costs, order totals, quantities, line totals, inventory values, and related calculations.

Initial observations included:

- No invalid negative product prices or costs in the tested data.
- No invalid order-item quantities in the tested data.
- No negative inventory quantities detected.
- 1,147 zero-price products.
- 1,147 zero-cost products.
- 349 zero-stock inventory records.
- Nine zero-price clickstream events in the initial 6,000-event snapshot.

Zero values were not automatically classified as invalid.

For example, zero inventory may represent a legitimate out-of-stock condition, while zero-price products may require business interpretation.

Financial consistency checks verified:

Order-item calculation:

```
LINE_TOTAL = QUANTITY × UNIT_PRICE
```

Order-total calculation:

```
ORDER_TOTAL = SUM(LINE_TOTAL)
```

The initial checks found no mismatches in the evaluated order-item calculations or order totals.

Decision: Preserve original numeric values and implement technical quality indicators and tests to detect future invalid or inconsistent records.

1.7 Referential Integrity

SQL validation was used to verify relationships between related source datasets.

The evaluated relationships included:

- Orders to Customers.
- Order Items to Orders.
- Order Items to Products.
- Inventory to Products.
- Clickstream Events to Customers.
- Clickstream Events to Products.

The initial profiling results showed no broken relationships in the tested datasets.

These checks established technical consistency between source entities without creating analytical joins or dimensional models.

1.8 Streaming Behavior and Session Analysis

The initial 6,000-event snapshot contained:

Event Type	Events
View	5,881
Cart	55
Purchase	64

No remove_from_cart events were observed in that initial sample.

The distribution was heavily dominated by view events.

These values describe the initial sample only and should be recalculated for subsequent streaming loads.

Session Identifier Analysis

The initial profiling discovered that the same USER_SESSION value could be associated with different users.

This means USER_SESSION alone was not sufficient as a globally unique session identifier.

Decision: Create a composite session identifier using USER_ID and USER_SESSION.

The derived SESSION_KEY identifies a user-session combination rather than an individual event.

Timestamp Analysis

Two different timestamps were identified:

- EVENT_TIME: Historical event timestamp from the source dataset.
- SIMULATED_AT: Timestamp associated with the Kafka replay simulation.

The initial historical events occurred in October 2019, while replay timestamps were recorded in October 2026.

SIMULATED_AT is not necessarily the actual Snowflake ingestion timestamp.

Decision: Preserve both timestamp fields separately to avoid mixing historical event time with replay activity.

1.9 Final Profiling Conclusions

The profiling results established the following requirements:

1. Preserve source records and original business meaning.
2. Standardize text values consistently.
3. Retain missing optional attributes with explicit quality indicators.
4. Distinguish zero values from invalid negative values.
5. Preserve financial calculations and validate their consistency.

6. Validate source relationships and required technical identifiers.
7. Introduce a composite session key to prevent session collisions between
- users.

1. Preserve EVENT_ID for individual event identification.
2. Maintain separate historical and replay timestamps.
3. Avoid automatic deletion of events based solely on matching payloads.

Final Assessment

The evaluated RAW datasets provided a suitable foundation for dbt Staging and Technical Data Quality transformations.

The identified requirements guided the cleaning, quality indicators, automated tests, and monitoring logic implemented in subsequent phases.

2. Data Staging & Cleaning

Technology: dbt Core 1.12.6, dbt-snowflake 1.12.4, Snowflake SQL, VS Code

Data Flow: Snowflake RAW → dbt STAGING Views

Status: Completed

2.1 Objective

The objective of the Data Staging & Cleaning section was to transform the six RAW source datasets into consistent and reusable staging models while preserving their original records and business meaning.

Transformations were developed in VS Code using SQL and dbt Core.

The actual SQL execution was performed inside Snowflake through the dbt- snowflake adapter.

All six staging models were configured as Snowflake Views rather than physically materialized tables.

2.2 Staging Architecture

The data flow is:

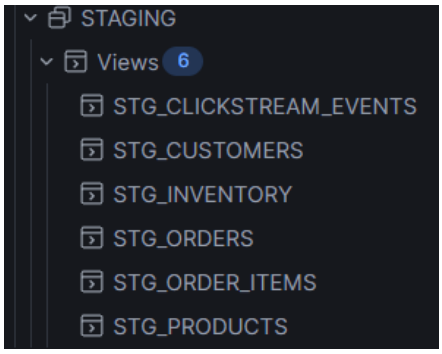
```
PostgreSQL OLTP / Kafka Clickstream → Snowflake RAW → dbt STAGING → Data Warehouse / Gold →
Analytics
```

Each staging model corresponds to one RAW source dataset.

The staging layer performs source-level cleaning and technical preparation.

Analytical joins, aggregations, dimensional models, and business KPIs remain outside the scope of this phase.

2.3 Implemented Staging Models



2.3.1 Customers — stg_customers

The customer staging model performs text standardization.

Implemented transformations include:

- TRIM for customer names.
- TRIM for country values.
- LOWER and TRIM for email addresses.
- Preservation of CUSTOMER_ID and registration timestamps.

The purpose is to produce consistent customer attributes without modifying customer identities or removing legitimate records.

2.3.2 Products — stg_products

The product staging model handles optional descriptive attributes and numerical quality indicators.

Implemented transformations include:

- Normalization of CATEGORY_CODE and BRAND using TRIM and NULLIF.
- Preservation of original missing values.
- CATEGORY_CODE_LABEL and BRAND_LABEL display fields using Unknown where appropriate.
- Indicators for missing category and brand information.
- Indicators for zero or invalid numerical values.

Missing descriptive attributes are not automatically replaced with invented product information.

2.3.3 Orders — stg_orders

The orders model standardizes order-status values.

Implemented transformations include:

- Preservation of ORDER_STATUS_RAW.
- UPPER and TRIM for the standardized ORDER_STATUS.
- Preservation of original order identifiers, customer references, totals, and timestamps.

This standardization improves consistency without introducing business-specific revenue recognition rules.

2.3.4 Order Items — stg_order_items

The order-items staging model preserves source-level purchase-line details.

It includes an indicator identifying mismatches between reported LINE_TOTAL and the calculated value of QUANTITY multiplied by UNIT_PRICE.

Original numeric values remain available for investigation.

2.3.5 Inventory — stg_inventory

The inventory staging model normalizes source text and adds operational indicators.

Implemented transformations include:

- Normalization of INVENTORY_ORIGIN.
- IS_OUT_OF_STOCK.
- IS_AT_OR_BELOW_REORDER_LEVEL.

Zero inventory is treated as a potentially legitimate operational condition rather than an automatic technical error.

2.3.6 Clickstream Events — stg_clickstream_events

The clickstream staging model prepares event data for downstream technical and analytical use.

Implemented transformations include:

- Preservation of EVENT_ID from the RAW source.
- LOWER and TRIM for EVENT_TYPE.
- Normalization of BRAND and CATEGORY_CODE.
- Creation of display labels for missing descriptive attributes.
- Preservation of EVENT_TIME.
- Preservation of SIMULATED_AT.
- Creation of a composite SESSION_KEY.
- Technical quality indicators for missing attributes and invalid values.

EVENT_ID: Identifies an individual event and is supplied by the upstream ingestion pipeline.

SESSION_KEY: Identifies a user-session combination. It is derived from USER_ID and USER_SESSION using SHA2-256.

Multiple events may share the same SESSION_KEY, but individual events should have distinct EVENT_ID values.

The staging model does not automatically deduplicate events.

2.4 Data Quality and Validation

Generic dbt tests were implemented to validate:

- Required identifiers.
- Primary-key uniqueness.
- Referential integrity.
- Accepted categorical values.
- Required clickstream timestamps and session fields.
- EVENT_ID completeness and uniqueness.

Independent Snowflake SQL was also used to inspect staging results, validate standardized values, and compare source and staging row counts.

2.5 Data Preservation Decisions

The following policies were applied:

- Missing product category and brand attributes were preserved.
- Unknown labels were provided separately from original NULL values.
- Zero-price and zero-stock records were retained for interpretation.
- Invalid-value indicators were added without automatically deleting records.
- EVENT_ID was preserved from the source.
- Identical event payloads were not automatically deleted.
- Historical event timestamps remained separate from replay timestamps.
- RAW tables were not modified by dbt transformations.

2.6 Materialization and Streaming Compatibility

All six models use Snowflake View materialization.

A View stores a SQL definition rather than a separate physical copy of the source rows.

New records committed to compatible RAW tables are therefore visible through the staging views when queried.

Repeated dbt run commands are not required merely because new source rows arrive.

However, dbt tests do not execute automatically when the source data changes.

2.7 Final Outcomes

the Data Staging & Cleaning section produced six consistent and reusable staging views.

The updated clickstream staging view successfully exposes EVENT_ID alongside the existing cleaning logic and composite SESSION_KEY.

The latest verified clickstream snapshot contained 150,000 events with no NULL or duplicate EVENT_ID values.

The staging models provide a prepared technical interface for downstream Data Warehouse and Analytics teams.

Final Status: COMPLETED

3. Technical Data Transformations & Advanced Data Quality

Technology: Snowflake, dbt Core, dbt-snowflake, SQL, Git, VS Code

Source Schema: RAW

Target Schema: STAGING

Status: Completed

3.1 Implementation Overview

the Technical Data Transformations & Advanced Data Quality section focused on strengthening technical data quality within the existing staging models.

The objective was to add diagnostic indicators and validation rules without changing source-level granularity or introducing business-specific analytical transformations.

The phase deliberately excluded:

- Cross-table analytical joins.
- Business aggregations.
- Fact and dimension modeling.
- Star-schema design.
- Revenue and conversion KPI calculations.
- Automatic removal of potentially legitimate events.

These responsibilities remain with the downstream Data Warehouse and Analytics teams.

3.2 Architecture and Scope

The architecture remains:

```
PostgreSQL OLTP + Kafka Replay → Snowflake RAW → dbt STAGING → Downstream Data Warehouse
```

All six staging models remain Snowflake Views.

Five staging models were enhanced with technical quality indicators, while the customer cleaning model was preserved.

No additional dimensional tables or business-specific marts were created.

3.3 Technical Transformations Implemented

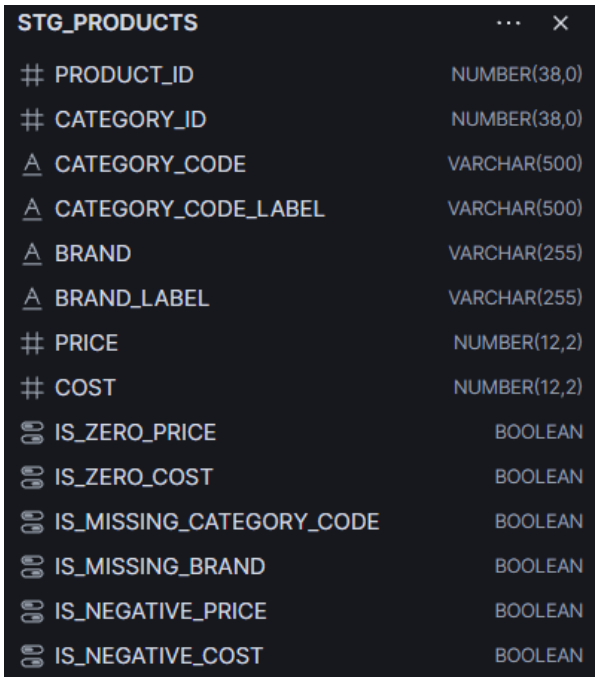
3.3.1 Products — stg_products

Technical quality indicators were introduced for:

- Missing category information.
- Missing brand information.
- Zero-price and zero-cost values.
- Invalid negative prices.
- Invalid negative costs.

Original nullable descriptive attributes remain available.

Missing brand or category information does not result in automatic record deletion.



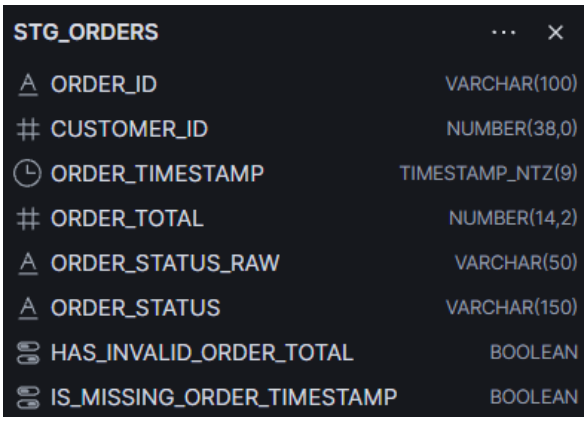
3.3.2 Orders — stg_orders

Technical indicators identify:

- Invalid order totals.
- Missing order timestamps.

Standardized order statuses are preserved alongside the original values.

No business interpretation of cancelled, refunded, shipped, or delivered orders is performed at this stage.



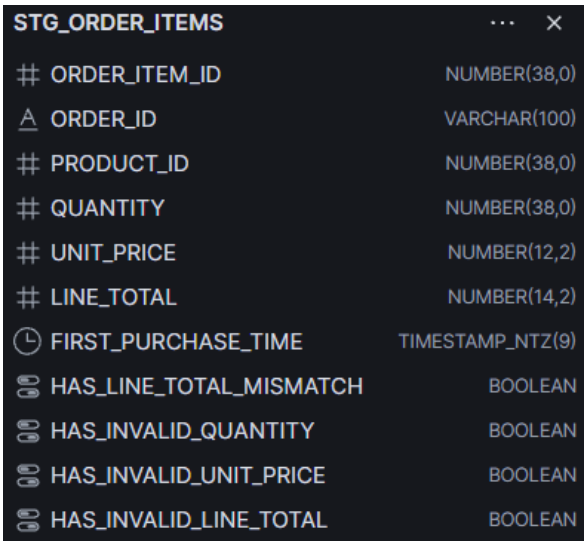
3.3.3 Order Items — stg_order_items

Technical indicators identify:

- Invalid quantities.
- Invalid unit prices.
- Invalid line totals.
- Mismatches between calculated and reported line totals.

The original order-item grain and financial values remain unchanged.

The indicators support detection of future source inconsistencies.



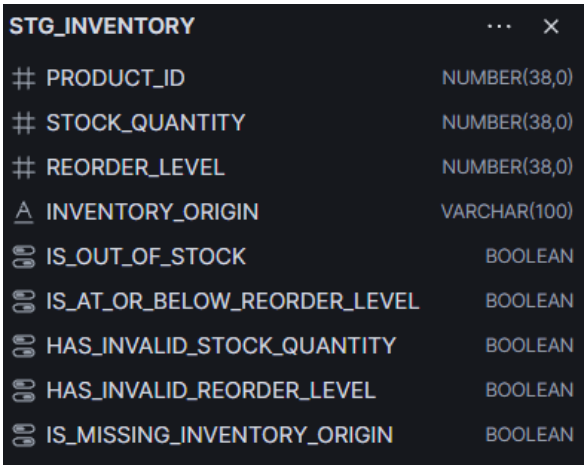
3.3.4 Inventory — stg_inventory

Technical indicators identify:

- Invalid stock quantities.
- Invalid reorder levels.

- Missing inventory-origin information.

Zero-stock values remain valid as they may represent legitimately out-of-stock products.



3.3.5 Clickstream Events — stg_clickstream_events

The clickstream model contains technical indicators identifying:

- Missing category information.
- Missing brand information.
- Zero-price events.
- Invalid event prices.
- Invalid session identifiers.
- Missing historical or replay timestamps.

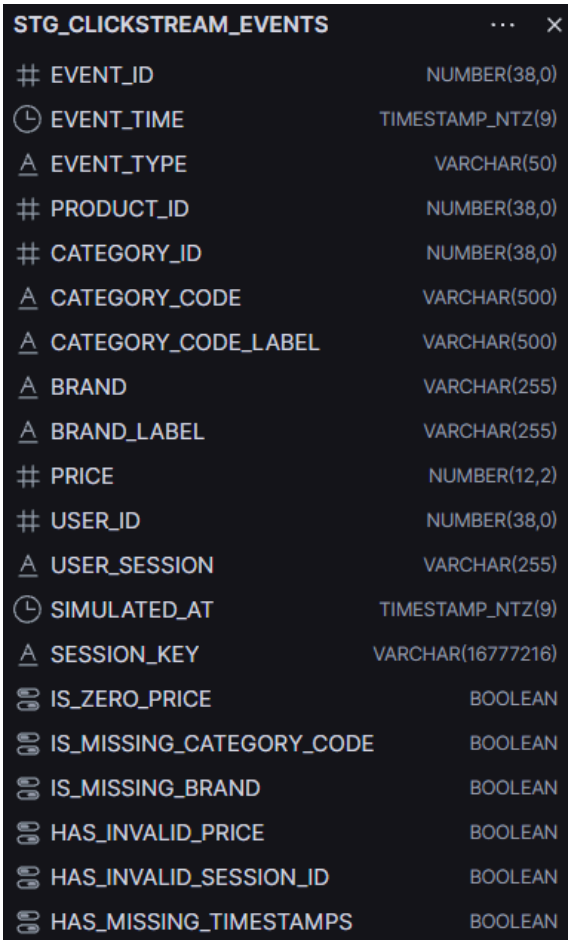
The source EVENT_ID is preserved as the individual event identifier.

The composite SESSION_KEY remains available to distinguish user-session combinations.

EVENT_ID and SESSION_KEY are not interchangeable:

- EVENT_ID identifies an individual event.
- SESSION_KEY identifies a session belonging to a particular user.

No automatic event deduplication or session-level aggregation is implemented.



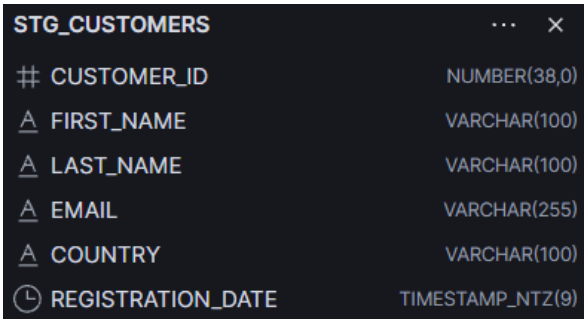
3.3.6 Customers — stg_customers

The existing customer transformation remains unchanged.

Additional automated checks validate:

- Email standardization.
- Registration-date completeness.

No customer records are automatically merged or removed.



3.4 Data Quality Automation

Data quality logic is implemented through two testing mechanisms.

Generic Tests

Defined in:

```
retail_dbt/models/staging/staging_models.yml
```

These tests validate uniqueness, non-null requirements, relationships, and accepted values.

The current implementation also validates EVENT_ID completeness and uniqueness.

Singular SQL Tests

Defined in:

```
retail_dbt/tests/technical_quality/
```

Seven custom SQL tests validate customer, product, order, order-item, inventory, and clickstream technical quality conditions.

The current project contains:

Test Type	Count
Generic Tests	30
Singular SQL Tests	7
Total Automated Tests Defined	37

3.5 dbt Execution and Validation

The initial complete implementation was validated through:

```
dbt build --select path:models/staging path:tests/technical_quality
```

Historical complete build result:

Metric	Result
Staging Models Built	6
Tests Executed	35
Total Operations	41
Passed	41
Warnings	0
Errors	0
Skipped	0

These results describe the complete build executed before the EVENT_ID tests were added.

Following the EVENT_ID update, the clickstream model was deployed and its selected tests were re-executed.

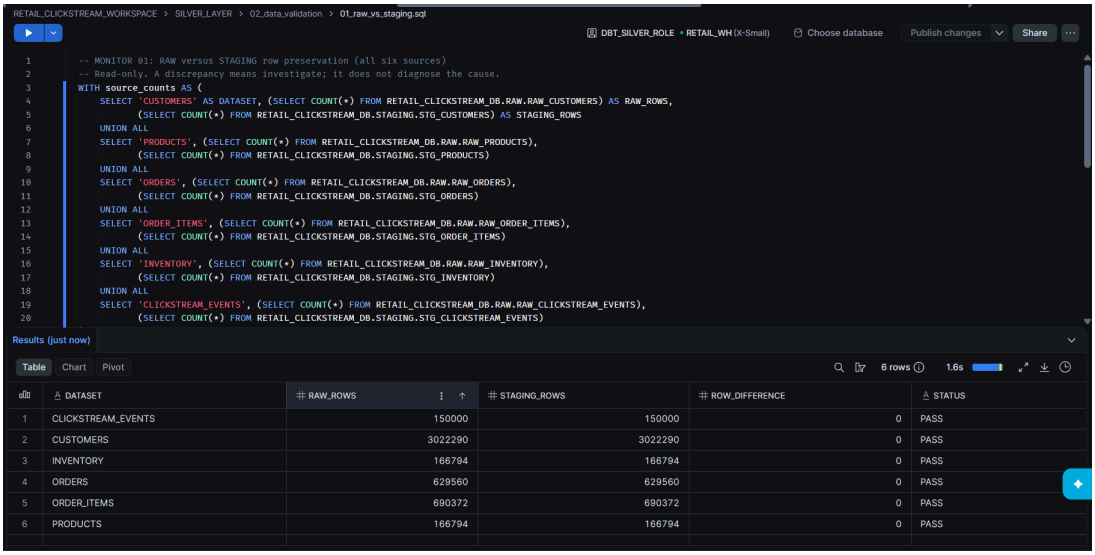
Latest verified clickstream testing:

Metric	Result
Tests Executed	12
Passed	12
Warnings	0
Errors	0
Skipped	0

The full 37-test suite has not yet been rerun as one complete execution following the latest EVENT_ID changes.

3.6 Snowflake Validation Results

Validation 1 — RAW vs STAGING Row Preservation



The original complete validation recorded:

Dataset	RAW Rows	STAGING Rows	Difference
Customers	3,022,290	3,022,290	0
Products	166,794	166,794	0
Orders	629,560	629,560	0
Order Items	690,372	690,372	0
Inventory	166,794	166,794	0
Clickstream Events	6,000	6,000	0

These figures represent the historical complete validation baseline.

The latest verified clickstream snapshot contains 150,000 events in both RAW and STAGING.

Validation 2 — Missing Attributes and Zero Values

Initial product findings:

Metric	Result
Missing Category	93,644
Missing Brand	43,613
Zero Price	1,147
Invalid Numeric Values	0

Initial 6,000-event clickstream findings:

Metric	Result
Missing Category	1,922
Missing Brand	792
Zero Price	9
Invalid Numeric Values	0

The clickstream measurements above are historical and require recalculation to describe the current 150,000-event snapshot.

Validation 3 — Critical Record Quality

The initial validation reported zero critical invalid records under the implemented rules for:

- Orders.
- Order Items.
- Inventory.
- Clickstream Events.

The latest selected clickstream tests subsequently passed successfully on the updated source data.

Validation 4 — Event Identity and Duplicate Analysis

The initial clickstream payload audit found one identical payload group with four occurrences.

This result was treated as an observation rather than definitive proof of duplicate ingestion.

The current implementation uses EVENT_ID as the primary event-identity field.

Latest verified EVENT_ID results:

Metric	Result
Total Events	150,000
Missing EVENT_ID Values	0
Distinct EVENT_ID Values	150,000
Duplicate EVENT_ID Rows	0

Payload-similarity analysis remains available as an optional diagnostic check.

No automatic event deletion is performed in staging.

Validation 5 — Timestamp Preservation

The initial 6,000-event validation confirmed matching historical and replay timestamps between RAW and STAGING.

The current model continues to preserve EVENT_TIME and SIMULATED_AT as separate source attributes.

These fields represent different time concepts and should not be substituted for one another.

3.7 Data Quality Policies

The implemented technical policies include:

1. Preserve source-level records.
2. Retain original descriptive attributes.
3. Distinguish missing, zero, and invalid values.
4. Use quality flags for record-level diagnostics.
5. Preserve EVENT_ID for event identification.
6. Preserve SESSION_KEY for user-session grouping.
7. Avoid unsupported automatic event deletion.
8. Maintain original and replay timestamps separately.
9. Avoid premature analytical or business transformations.

3.8 Project Organization

The dbt project is organized as follows:

```
retail_dbt/
├── dbt_project.yml
├── README.md
├── requirements.txt
├── models/
│   └── staging/
│       ├── sources.yml
│       ├── staging_models.yml
│       ├── stg_customers.sql
│       ├── stg_products.sql
│       ├── stg_orders.sql
│       ├── stg_order_items.sql
│       ├── stg_inventory.sql
│       └── stg_clickstream_events.sql
└── tests/
    └── technical_quality/
└── 7 custom singular SQL tests
```

Standalone SQL scripts for profiling, validation, monitoring, and orchestration handoff are maintained separately under:

```
SQL_scripts/silver/
```

The dbt project remains the source of truth for staging transformations.

3.9 Final Deliverables

The technical transformations and data quality implementation provides:

1. Six standardized staging views.
2. Preserved source-level grain.
3. Technical quality flags for missing, zero, and invalid values.
4. A source-provided EVENT_ID in clickstream staging.
5. A composite SESSION_KEY.
6. Repeatable automated dbt tests.
7. Referential and key-integrity validation.
8. Independent Snowflake SQL checks.
9. Timestamp preservation.
10. Version-controlled implementation and technical documentation.

Conclusion

The technical transformations and data quality implementation established a technically validated staging layer that can be consumed by downstream dimensional modeling.

The initial complete build succeeded, and the updated clickstream implementation subsequently passed all 12 selected tests after the EVENT_ID enhancement.

4. Data Quality Assurance & Automated Testing

Technology: dbt Core, dbt-snowflake, Snowflake SQL

Status: Implemented — Updated Clickstream Tests Passed

4.1 Objective

The objective of the Data Quality Assurance & Automated Testing section was to establish a repeatable Data Quality Assurance framework for the six staging datasets.

The framework validates technical data integrity before the prepared source data is consumed by downstream warehouse models.

Quality tests are designed to identify invalid records rather than automatically remove or overwrite them.

4.2 Implementation

Automated quality tests are implemented in two locations.

Generic Data Tests

Location:

```
retail_dbt/models/staging/staging_models.yml
```

Generic tests cover four main categories:

Not Null

Ensures required fields and technical identifiers are present.

Unique

Ensures identifiers expected to be unique are not repeated.

Relationships

Validates references between related datasets.

Accepted Values

Ensures categorical fields contain allowed values.

The current configuration contains 30 generic tests.

Singular SQL Tests

Location:

```
retail_dbt/tests/technical_quality/
```

Seven custom tests validate:

1. Customer email standardization.
2. Customer registration-date completeness.
3. Product numeric validity.
4. Order validity.
5. Order-item validity and financial consistency.
6. Inventory numeric validity.
7. Clickstream technical validity.

A singular SQL test returns records violating its validation rule.

A test passes when zero violating rows are returned.

4.3 EVENT_ID Integrity Tests

The clickstream staging model preserves the EVENT_ID supplied by the ingestion pipeline.

Two additional generic tests are implemented:

EVENT_ID — not_null

Ensures every clickstream event contains an EVENT_ID value.

EVENT_ID — unique

Ensures no EVENT_ID occurs more than once in the staging result.

These tests complement existing validations for:

- EVENT_TIME.
- SIMULATED_AT.
- EVENT_TYPE.
- USER_ID.
- PRODUCT_ID.
- USER_SESSION.
- Customer and product relationships.
- Technical clickstream quality conditions.

EVENT_ID uniqueness verifies the current dataset but does not independently guarantee exactly-once delivery across Kafka retries or replay sessions.

4.4 Execution Commands

From the dbt project directory:

```
dbt build --select path:models/staging path:tests/technical_quality
```

This command builds selected models and executes their associated tests.

To rerun tests without rebuilding unchanged staging views:

```
dbt test --select path:models/staging path:tests/technical_quality
```

For clickstream-specific validation:

```
dbt test --select stg_clickstream_events invalid_clickstream_records
```

4.5 Validation Results

```
18:56:45  Finished running 37 data tests in 0 hours 0 minutes and 7.65 seconds (7.65s).
18:56:46
18:56:46  Completed successfully
18:56:46
18:56:46  Done. PASS=37 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=37
```

The original complete implementation passed six staging model builds and 35 automated tests, resulting in 41 successful dbt operations.

After introducing EVENT_ID validation, the project contains:

Metric	Current Configuration
Staging Models	6
Generic Tests	30

Metric	Current Configuration
Singular SQL Tests	7
Total Automated Tests	37

Latest verified updated clickstream execution:

Metric	Result
Tests Executed	12
Passed	12
Warnings	0
Errors	0
Skipped	0

The updated clickstream EVENT_ID checks also returned:

- 150,000 total events.
- 150,000 distinct EVENT_ID values.
- Zero NULL EVENT_ID values.
- Zero duplicate EVENT_ID rows.

Latest Full Data Quality Test Execution

The complete dbt data quality test suite was successfully executed following the latest EVENT_ID integration.

Metric	Result
Generic Tests	30
Custom Singular Tests	7
Total Tests Executed	37
Tests Passed	37
Warnings	0
Errors	0
Execution Time	7.65 seconds

Final Status: SUCCESS — All 37 data quality tests passed successfully.

This confirms that all implemented staging data quality rules passed at the time of execution.

4.6 Quality Assurance Principles

The framework follows several technical principles:

- Validate required identifiers.
- Preserve legitimate records.
- Detect rather than silently overwrite invalid values.
- Separate source data quality from business interpretation.

- Maintain referential integrity.
- Validate financial calculations.
- Preserve event identity.
- Keep automated tests repeatable.

Tests do not create database primary-key or foreign-key constraints merely by being defined in dbt.

They perform SQL validation against the data.

4.7 Outcome

the Data Quality Assurance & Automated Testing section established a repeatable and version-controlled Data Quality Assurance framework.

The implementation provides quality gates for source-level staging datasets and can be integrated into scheduled or trigger-based orchestration.

The tests require explicit execution or scheduling; they are not triggered automatically whenever Kafka or batch ingestion inserts new records.

Final Status: IMPLEMENTED — LATEST CLICKSTREAM VALIDATION PASSED 12/12

5. Technical Data Monitoring & Observability

Technology: Snowflake SQL, Snowsight Workspaces, dbt, GitHub

Monitoring Type: Manual / On-Demand

Status: Completed — Monitoring Logic Prepared and Core Checks Validated

5.1 Objective

The objective of the Technical Data Monitoring & Observability section was to establish lightweight technical monitoring for the Silver staging layer.

The monitoring logic helps identify ingestion discrepancies, source-to-staging inconsistencies, invalid records, missing attributes, event-identity problems, and changes in clickstream behavior.

This phase does not implement automated orchestration, scheduling, alerts, or business-intelligence dashboards.

5.2 Monitoring Architecture

The monitoring layer reads from Snowflake RAW tables and STAGING views using read-only SQL queries.

The monitoring workflow is:

```
Snowflake RAW / STAGING → SQL Monitoring Queries → Technical Results → Investigation /
Orchestration Handoff
```

The queries provide current metrics when executed.

They do not automatically store historical monitoring results or run continuously in the background.

5.3 Core Monitoring Checks

Six core SQL monitoring checks were implemented.

Monitor	Purpose
01 — Row Count Reconciliation	Compare RAW and STAGING row counts across all six datasets.
02 — Streaming Checkpoint	Monitor clickstream event counts and historical/replay timestamps.
03 — Technical Quality Summary	Identify records violating technical quality conditions.
04 — Missing Attributes and Zero Prices	Measure missing descriptive attributes and zero-price indicators.

Monitor	Purpose
05 — Event ID Integrity	Detect NULL and duplicate EVENT_ID values and count distinct events.
06 — Event Type Distribution	Examine the distribution of clickstream event types.

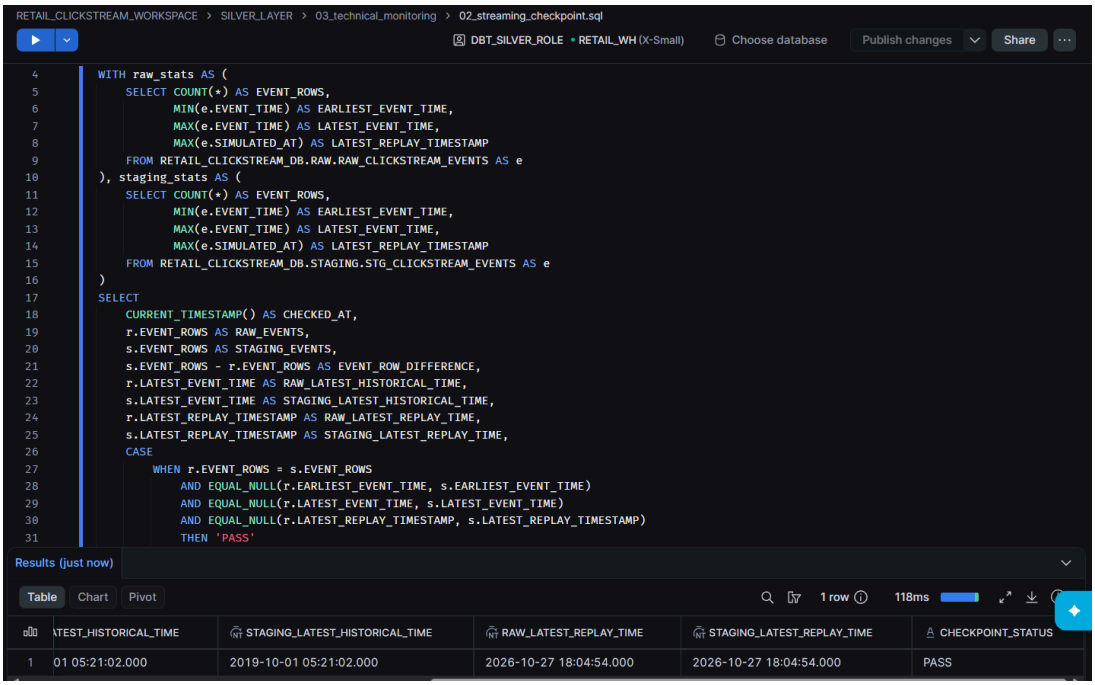
Monitor 01 — Row Count Reconciliation

Compares the number of rows in each RAW source with the corresponding STAGING view.

The expected behavior is that source and staging counts remain aligned because the staging models preserve source-level grain.

A discrepancy indicates that additional investigation is required.

Monitor 02 — Streaming Checkpoint

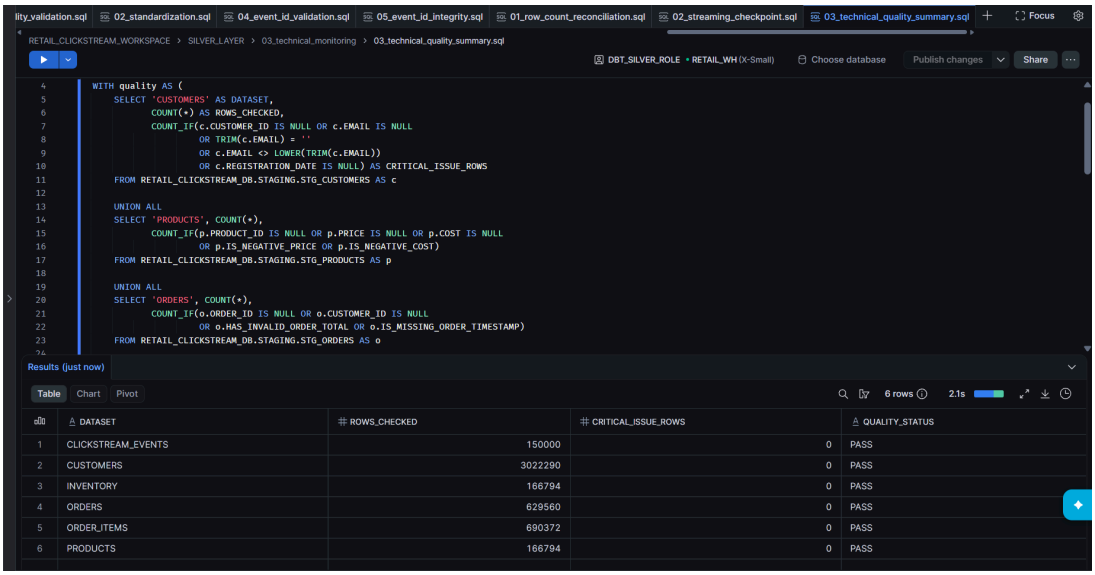


Tracks clickstream event counts and the latest historical and replay timestamps.

The monitor helps determine whether new event records are visible in the Snowflake layers.

It does not independently prove exactly-once Kafka delivery or real-time ingestion latency.

Monitor 03 — Technical Quality Summary

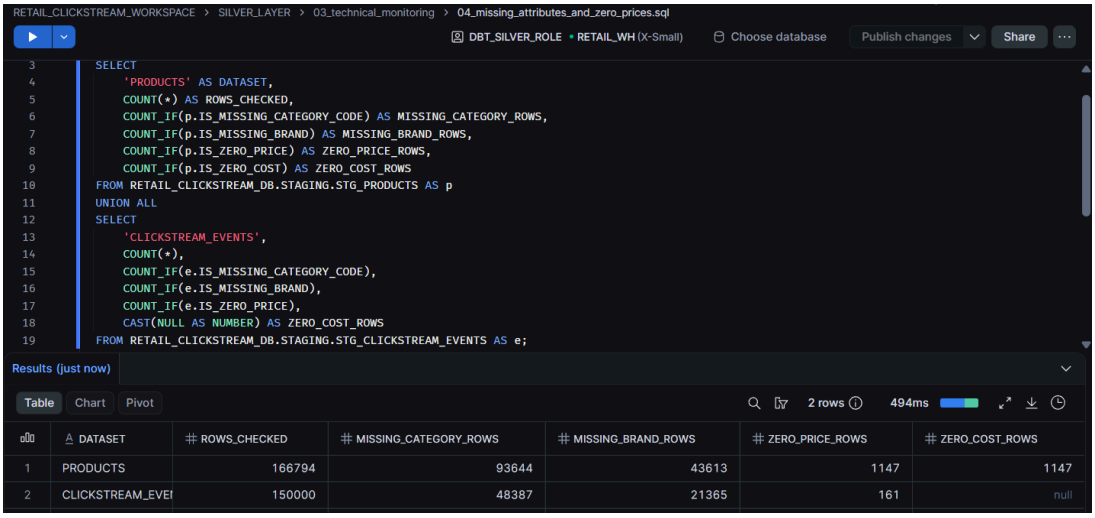


Reports counts of records violating selected technical quality rules.

Examples include invalid order totals, invalid quantities, invalid stock values, missing timestamps, and invalid clickstream fields.

These metrics complement the Pass/Fail results of dbt tests.

Monitor 04 — Missing Attributes and Zero Prices

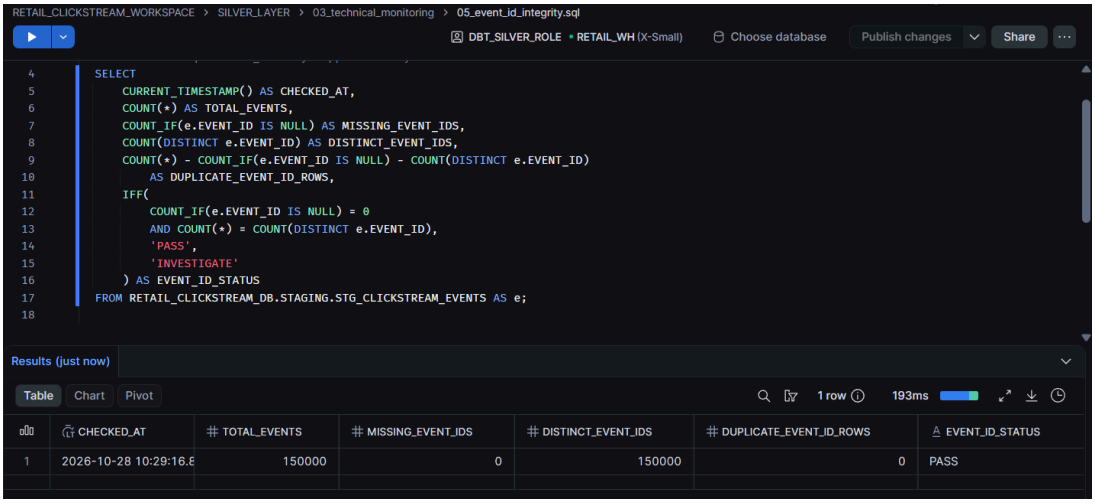


Tracks missing category and brand attributes and zero-price values.

These conditions may be analytically significant without necessarily being technical errors.

The monitoring queries make such conditions visible for investigation and downstream interpretation.

Monitor 05 — Event ID Integrity



Uses EVENT_ID to inspect event-level identity.

The primary metrics are:

- Total event count.
- Missing EVENT_ID count.
- Distinct EVENT_ID count.
- Duplicate EVENT_ID row count.

The latest verified snapshot showed:

Metric	Result
Total Events	150,000
Missing EVENT_ID Values	0
Distinct EVENT_ID Values	150,000
Duplicate EVENT_ID Rows	0

A unique identifier in one load does not, by itself, prove that the ingestion pipeline preserves the same event identity across retries and replay operations.

Monitor 06 — Event Type Distribution

Examines the relative distribution of clickstream event types, including view, cart, purchase, and other accepted events.

The purpose is to detect unexpected changes in the incoming data.

This check is technical monitoring rather than final business funnel or conversion analysis.

5.4 Optional Diagnostic Checks

Additional SQL scripts are available for:

Payload Similarity Audit

Identifies events with matching payload fields.

This is a diagnostic check and does not conclusively establish that two events are the same logical event.

Snowflake Query History

Supports inspection of query execution information and Snowflake activity when required.

These additional scripts are optional and do not replace the six core monitoring checks.

5.5 Workspace Organization

The SQL files were organized into separate functional folders.

```
SILVER_LAYER/
│
├── 01_data_profiling/
│
├── 02_data_validation/
│
├── 03_technical_monitoring/
│
└── 04_orchestration_handoff/
```

The folder structure is available in the shared Snowflake workspace and is version-controlled in the project repository under:

```
SQL_scripts/silver/
```

01_data_profiling

Contains source inventory, completeness, uniqueness, numeric consistency, referential integrity, streaming behavior, and EVENT_ID profiling SQL.

02_data_validation

Contains RAW versus STAGING comparisons, standardization checks, technical quality validation, and EVENT_ID validation SQL.

03_technical_monitoring

Contains reusable technical monitoring queries and optional diagnostics.

04_orchestration_handoff

Contains the Silver readiness query and a technical runbook for the orchestration teammate.

The runbook describes:

- Existing staging views.
- Quality gates.
- Commands for executing dbt tests.
- Source-to-staging update behavior.

- Downstream Gold dependencies.
- Triggering and scheduling responsibilities.

5.6 Established Monitoring Baseline

The initial complete monitoring baseline included:

- Matching RAW and STAGING row counts across six source datasets.
- Zero critical invalid records in the evaluated quality categories.
- Identical historical and replay timestamps between the initial RAW and STAGING clickstream snapshots.
- One repeated payload group observed in the original 6,000-event sample.

Following the event-identity update, the latest verified clickstream baseline contains:

- 150,000 RAW events.
- 150,000 STAGING events.
- Zero missing EVENT_ID values.
- Zero duplicate EVENT_ID rows.
- Twelve successful selected clickstream dbt tests.

All monitoring measurements are point-in-time observations rather than permanent source-data guarantees.

5.7 Streaming and Update Behavior

The Silver layer uses dbt-managed Snowflake Views.

New records successfully committed to the RAW clickstream table become available through the corresponding staging view when queried.

The staging views do not require a recurring dbt run simply to display newly inserted records.

However:

- Kafka Producer success does not guarantee successful Snowflake ingestion.
- Materialized Gold tables may require scheduled or triggered refresh.
- Dashboards require their own data-query refresh.
- dbt tests require independent execution or scheduling.
- Stable EVENT_ID generation across replays remains an upstream ingestion responsibility.

5.8 Operational Responsibilities

Data Engineering — Silver Layer

Responsibilities include:

- Maintaining dbt staging models.
- Defining technical quality indicators.

- Maintaining automated dbt tests.
- Maintaining technical monitoring SQL.
- Investigating anomalies.
- Providing validated staging interfaces.

Data Warehouse / Gold

Responsibilities include:

- Analytical joins.
- Fact and dimension modeling.
- Data marts.
- Business aggregation rules.
- Downstream model refresh strategy.

Orchestration & Analytics

Responsibilities include:

- Scheduling ingestion-dependent tasks.
- Executing dbt tests according to the orchestration strategy.
- Implementing alerts and operational notifications.
- Coordinating Gold refreshes.
- Developing dashboards and business insights.

5.9 Orchestration Handoff

The primary source of transformation logic is the version-controlled dbt project:

```
retail_dbt/
```

The reusable standalone SQL checks are maintained under:

```
SQL_scripts/silver/
```

The orchestration teammate can execute the appropriate dbt tests after ingestion updates.

If model definitions change, the modified models must be deployed before relying on their new definitions.

If only source rows change and the view definitions remain compatible, the existing Snowflake Views can be queried without rebuilding them.

Failed tests should be investigated before downstream data is treated as approved, according to the team's agreed operational policy.

The Silver implementation does not itself include an Airflow DAG, automatic Snowflake Task, or scheduled alerting workflow.

5.10 Final Outcome

the Technical Data Monitoring & Observability section established reusable technical monitoring and validation capabilities for the Silver layer.

The monitoring queries support source-to-staging reconciliation, event-identity verification, technical quality investigation, and downstream operational handoff.

The implementation preserves clear ownership boundaries between Data Engineering, Data Warehouse Modeling, and Orchestration / Analytics.

Final Status: COMPLETED — MANUAL MONITORING AND ORCHESTRATION HANDOFF PREPARED

Overall Silver Layer Conclusion

The Retail Clickstream Analytics Silver Layer provides a source-preserving, standardized, and technically validated interface between the ingestion pipeline and downstream warehouse models.

The implementation includes:

- Six dbt staging views.
- Source-level cleaning and standardization.
- Technical quality flags.
- EVENT_ID preservation and integrity checks.
- Composite user-session identification.
- Thirty generic dbt tests.
- Seven custom singular SQL tests.
- Reusable profiling and validation queries.
- Technical monitoring SQL.
- Snowflake workspace organization.
- GitHub version control.
- Documentation and orchestration handoff.

The latest verified clickstream snapshot contained 150,000 events with zero NULL or duplicate EVENT_ID values, and all 12 selected clickstream tests passed.

The complete current project defines 37 automated data tests. A new full-suite execution would be required to report updated results for all 37 tests together.

The Silver layer is designed to support newly ingested RAW records through Snowflake Views without repeated transformation runs for every ingestion batch.

Downstream fact/dimension modeling, business calculations, dashboard refresh, and orchestration remain responsibilities of the respective team members.

Gold Layer Implementation — dbt and Snowflake

1. Local dbt Configuration

The Gold Layer was implemented in the existing repository under:

```
retail-clickstream-analytics/retail_gold_dbt
```

The dbt project uses Python 3.11.9, dbt Core 1.12.6, and the dbt Snowflake adapter 1.12.4.

The Snowflake connection profile is configured in:

```
C:\Users\IT.SHOP\.dbt\profiles.yml
```

The active profile is gold_layer , with the dev target.

Configuration	Value
Account	xw44152.eu-central-2.aws
User	GOLD_USER
Role	DBT_SILVER_ROLE
Database	RETAIL_CLICKSTREAM_DB
Warehouse	RETAIL_WH
Target schema	MARTS
Threads	4

The Snowflake password is supplied through the RETAIL_SNOWFLAKE_PASSWORD environment variable rather than being stored directly in the profile.

dbt project settings

The default materialization settings in dbt_project.yml are:

```yaml
models:
retail_gold_dbt:
dimensions:
+materialized: table
facts:
+materialized: table
```

Each of the six models overrides the default with its own incremental MERGE configuration.

2. Staging Source Definitions

The source definitions are maintained in models/sources.yml .

Source	Used by
STG_CUSTOMERS	dim_customer
STG_ORDERS	dim_date, fact_order_items
STG_PRODUCTS	dim_product
STG_INVENTORY	dim_inventory
STG_ORDER_ITEMS	fact_order_items
STG_CLICKSTREAM_EVENTS	fact_clickstream_events

All sources belong to the RETAIL_CLICKSTREAM_DB.STAGING schema.

The model SQL uses source() references to access the staging data.

3. Dimension Models

The dimension models are stored in models/dimensions/ .

3.1 DIM_CUSTOMER

File: models/dimensions/dim_customer.sql

Source: STG_CUSTOMERS

Configured unique key: CUSTOMER_ID

Columns:

```
CUSTOMER_ID
FIRST_NAME
LAST_NAME
EMAIL
COUNTRY
REGISTRATION_DATE
```

The model selects the required customer attributes directly from staging and uses CUSTOMER_ID to match records during MERGE operations.

3.2 DIM_DATE

File: models/dimensions/dim_date.sql

Source: STG_ORDERS

Configured unique key: ORDER_TIMESTAMP

Columns:

```
ORDER_TIMESTAMP
FULL_DATE
YEAR
QUARTER
MONTH
DAY
DAY_OF_WEEK
MONTH_NAME
```

The model derives calendar attributes from the order timestamp using Snowflake date functions. SELECT DISTINCT removes duplicate timestamps from the query result, and null order timestamps are excluded.

The configured key remains ORDER_TIMESTAMP , as agreed for the Gold schema.

3.3 DIM_PRODUCT

File: models/dimensions/dim_product.sql

Source: STG_PRODUCTS

Configured unique key: PRODUCT_ID

Columns:

```
PRODUCT_ID
CATEGORY_ID
CATEGORY_CODE_LABEL
BRAND_LABEL
PRICE
COST
```

The model selects the prepared product attributes from staging. PRODUCT_ID is used to match existing product records during MERGE operations.

3.4 DIM_INVENTORY

File: models/dimensions/dim_inventory.sql

Source: STG_INVENTORY

Configured unique key: PRODUCT_ID

Columns:

```
PRODUCT_ID
STOCK_QUANTITY
REORDER_LEVEL
INVENTORY_ORIGIN
IS_OUT_OF_STOCK
IS_AT_OR_BELOW_REORDER_LEVEL
HAS_INVALID_STOCK_QUANTITY
HAS_INVALID_REORDER_LEVEL
IS_MISSING_INVENTORY_ORIGIN
```

The model selects inventory quantities, replenishment thresholds, and inventory quality indicators from staging. The configured key is PRODUCT_ID , assuming one applicable inventory record per product in the staging source.

Inventory remains a dimension in the agreed schema.

4. Fact Models

The fact models are stored in models/facts/ .

4.1 FACT_ORDER_ITEMS

File: models/facts/fact_order_items.sql

Sources: STG_ORDER_ITEMS and STG_ORDERS

Grain: One row per unique (ORDER_ID, PRODUCT_ID) combination.

Configured composite unique key: ORDER_ID + PRODUCT_ID

Columns:

```
ORDER_ID
PRODUCT_ID
CUSTOMER_ID
ORDER_TIMESTAMP
ORDER_STATUS
QUANTITY
UNIT_PRICE
LINE_TOTAL
FIRST_PURCHASE_TIME
ORDER_TOTAL
```

The model joins order items to order headers using ORDER_ID to bring customer, timestamp, status, and order-level attributes into the fact table.

4.2 FACT_CLICKSTREAM_EVENTS

File: models/facts/fact_clickstream_events.sql

Source: STG_CLICKSTREAM_EVENTS

Grain: One row per recorded clickstream event.

Configured unique key: EVENT_ID

Columns:

```
EVENT_ID
EVENT_TIME
EVENT_TYPE
PRODUCT_ID
CATEGORY_ID
BRAND
USER_ID
USER_SESSION
SESSION_KEY
IS_ZERO_PRICE
IS_MISSING_CATEGORY_CODE
HAS_INVALID_PRICE
HAS_INVALID_SESSION_ID
HAS_MISSING_TIMESTAMPS
```

The model selects event attributes and quality indicators directly from staging.

EVENT_ID is the numeric identifier used to match existing events during MERGE operations.

5. Incremental MERGE Configuration

All six models use the following configuration pattern:

```
{{
    config(
        materialized='incremental',
        incremental_strategy='merge',
        unique_key='YOUR_UNIQUE_KEY'
    )
}}
```

The actual unique key differs by model.

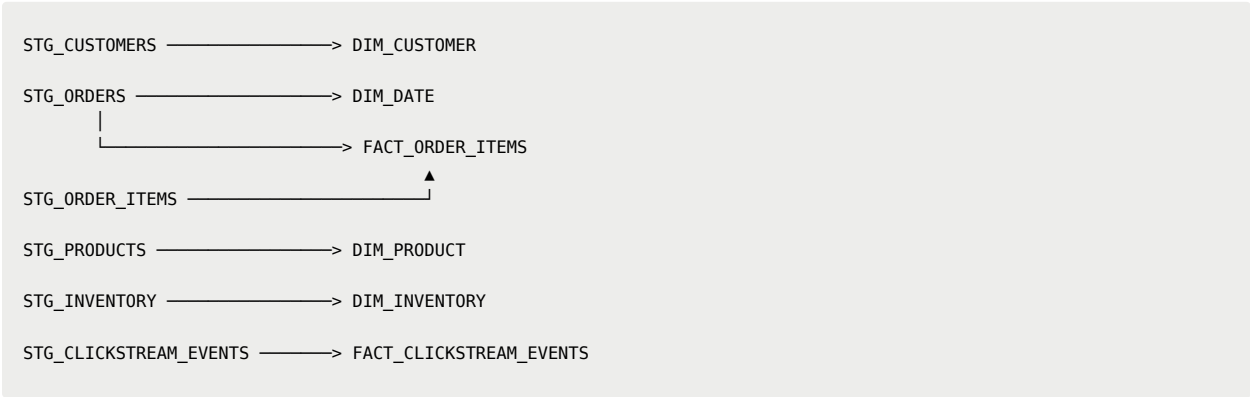
Model	Unique key
dim_customer	CUSTOMER_ID
dim_date	ORDER_TIMESTAMP
dim_product	PRODUCT_ID
dim_inventory	PRODUCT_ID
fact_order_items	ORDER_ID, PRODUCT_ID
fact_clickstream_events	EVENT_ID

The MERGE strategy matches source rows to existing target rows using the configured key. Matching records can be updated, while new keys can be inserted.

This configuration supports updating existing Gold records and inserting new records during incremental runs.

The current model queries read the full staging sources on each run. They do not yet apply incremental source filters to restrict processing to only new or changed source records. Therefore, incremental materialization does not necessarily eliminate full staging-source scans.

6. Model Dependencies



7. Compilation and Execution

All commands were run from the retail_gold_dbt directory with the .venv_gold environment active.

Compile the first five models

```
dbt compile --select dim_customer dim_date dim_product dim_inventory fact_order_items
```

Result: successful compilation.

Run the first five models

```
dbt run --select dim_customer dim_date dim_product dim_inventory fact_order_item
s
```

Result: all five models completed successfully, with no errors or warnings.

Compile the clickstream model

```
dbt compile --select fact_clickstream_events
```

Result: successful compilation. The compiled SQL resolved the expected staging source and event columns.

Run the clickstream model

```
dbt run --select fact_clickstream_events
```

Result: successful execution, processing 150,000 rows.

8. Clickstream Validation

Total rows, unique event IDs, and null IDs

```
SELECT
    COUNT(*) AS TOTAL_ROWS,
    COUNT(DISTINCT EVENT_ID) AS UNIQUE_EVENT_IDS,
    COUNT_IF(EVENT_ID IS NULL) AS NULL_EVENT_IDS
FROM RETAIL_CLICKSTREAM_DB.MARTS.FACT_CLICKSTREAM_EVENTS
```

Results:

Metric	Result
Total rows	150,000
Unique event IDs	150,000
Null event IDs	0

Duplicate event ID check

```
SELECT
    EVENT_ID,
```

```
    COUNT(*) AS ROW_COUNT
FROM RETAIL_CLICKSTREAM_DB.MARTS.FACT_CLICKSTREAM_EVENTS
GROUP BY EVENT_ID
HAVING COUNT(*) > 1
```

The query returned no rows.

Together, these results confirm that the current clickstream fact table contains 150,000 non-null, unique event IDs and no duplicate event IDs were found.

9. Schema Inspection

The deployed column definitions can be inspected directly in Snowflake.

Example:

```
DESCRIBE TABLE RETAIL_CLICKSTREAM_DB.MARTS.DIM_CUSTOMER;
```

Replace DIM_CUSTOMER with any of the other five table names to inspect their columns and data types.

10. dbt Documentation Generation

Model and column descriptions can be maintained in a YAML file under the models directory, for example models/gold_models.yml .

Once the descriptions are saved, generate the dbt documentation artifacts with:

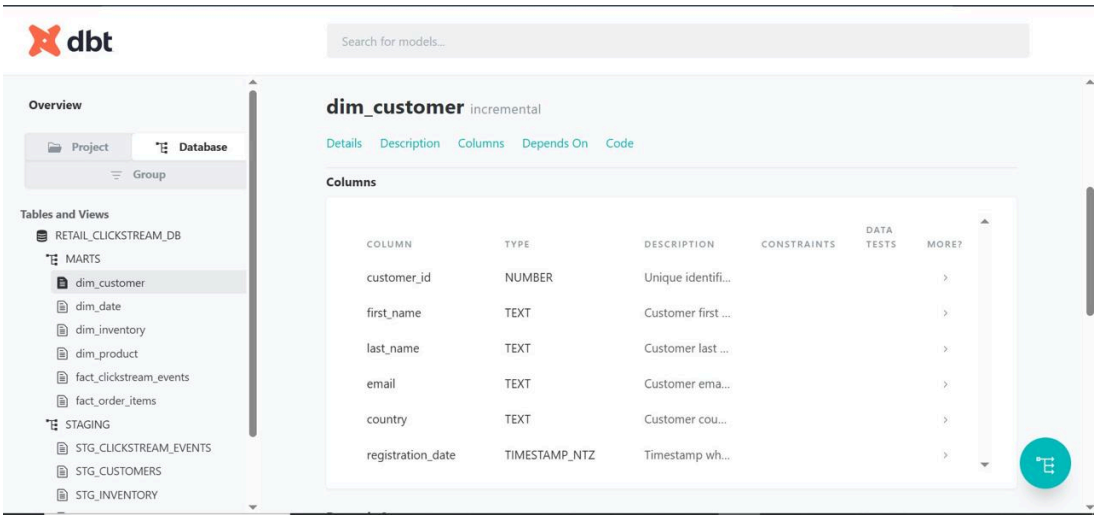
```
dbt docs generate
```

Start the local documentation site with:

```
dbt docs serve
```

Open the local URL printed in the terminal, commonly http://localhost:8080 .

The documentation interface provides a browsable view of model descriptions, column metadata, source information, and lineage.



11. Gold Layer Lineage Graph



12. Metric Classification Notes

The following considerations apply when the Gold tables are used in analytical reports.

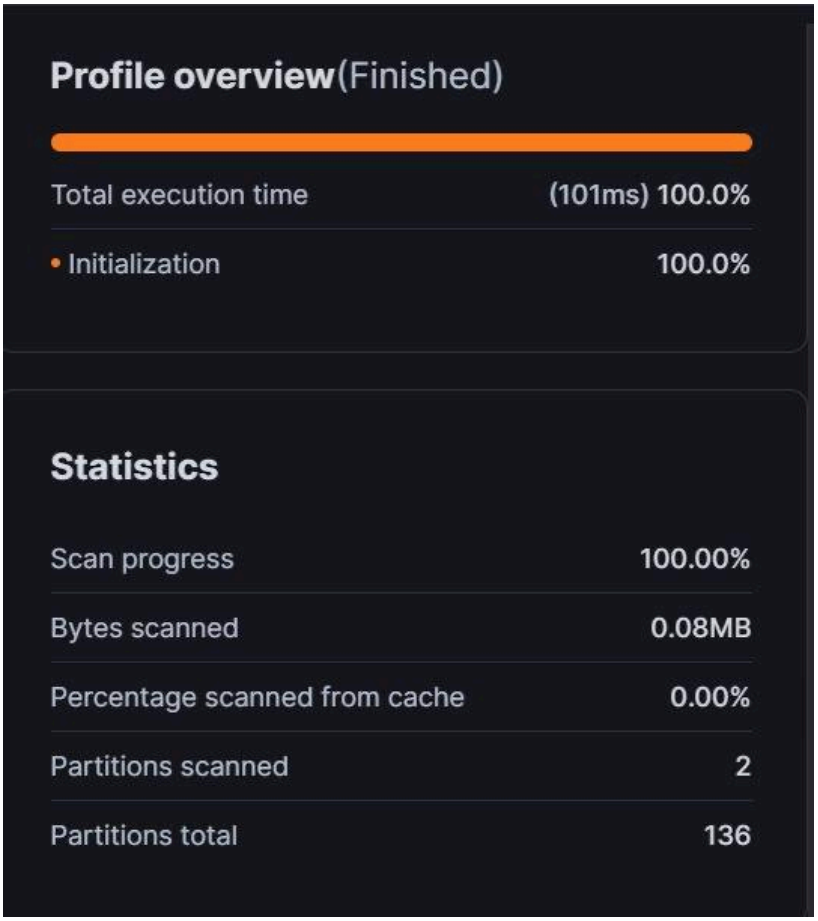
Metric or attribute	Classification	Aggregation consideration
QUANTITY	Additive	Can generally be summed across products, customers, and time
LINE_TOTAL	Additive, subject to business rules	Account for order statuses, cancellations, and returns when defining sales
ORDER_TOTAL	Conditionally additive	Can repeat across order-item rows; avoid summing repeated values directly
UNIT_PRICE	Non-additive	Use an appropriate average or quantity-weighted average
DIM_PRODUCT.PRICE	Non-additive attribute	Do not sum product prices across products
STOCK_QUANTITY	Semi-additive over time	Can be summed across products for a consistent point in time, but not across snapshots

Metric or attribute	Classification	Aggregation consideration
Distinct customers, orders, and sessions	Non-additive counts	Use distinct-count calculations
Clickstream event count	Additive	Sum events when each event is represented once

13. Snowflake Data Partitioning

Snowflake automatically manages data storage and micro-partitioning. When data is loaded into a table, Snowflake organizes it into micro-partitions and maintains metadata to help prune unnecessary data during queries. Unlike traditional databases, you do not need to manually create partitions for standard tables.

Snowflake also supports automatic clustering when needed for tables with large volumes of data or queries that may benefit from improved data organization. For this implementation, Snowflake's automatic micro-partitioning handles the physical data organization, so no manual partitioning was configured



Time Travel and Slowly Changing Dimensions

Snowflake Time Travel

Snowflake Time Travel allows users to access historical data from a table within its configured retention period. It supports querying previous versions of data and recovering data that was accidentally modified

or deleted.

For example, a previous version of a table can be queried using AT (OFFSET ⇒ ...) :

```
SELECT *
FROM RETAIL_CLICKSTREAM_DB.MARTS.DIM_CUSTOMER
AT (OFFSET => -60*5);
```

This query retrieves the table's state from approximately five minutes earlier, provided that the data is within the retention period

Slowly Changing Dimensions (SCD)

Slowly Changing Dimensions manage changes to dimension attributes over time.

- Type 1 — Overwrite: Updates an existing value without retaining its previous value in the dimension table. In this implementation, the dimension models use incremental MERGE operations to update matching records.
- Type 2 — Historical tracking: Retains previous versions of records, usually by adding effective start and end dates and a current-record indicator. This requires additional historical-tracking logic and is not implemented in the current models.

Implementation note: The current dbt models use incremental MERGE configurations. This supports updating matching records

SCD Type 2 Implementation for DIM_INVENTORY

Inventory Snapshot Configuration

A dbt snapshot named inventory_snapshot was created in snapshots/inventory_snapshot.sql .

The snapshot reads inventory records from STG_INVENTORY and detects changes using the check strategy. It monitors the following attributes:

```
STOCK_QUANTITY
REORDER_LEVEL
INVENTORY_ORIGIN
IS_OUT_OF_STOCK
IS_AT_OR_BELOW_REORDER_LEVEL
HAS_INVALID_STOCK_QUANTITY
HAS_INVALID_REORDER_LEVEL
IS_MISSING_INVENTORY_ORIGIN
```

When a monitored attribute changes between snapshot executions, dbt preserves the previous version and records a new version. The snapshot maintains the DBT_VALID_FROM and DBT_VALID_TO metadata columns to track the validity period of each version.

Snapshot Storage

The snapshot is stored in the existing MARTS schema:

```
RETAIL_CLICKSTREAM_DB.MARTS.INVENTORY_SNAPSHOT
```

The existing schema was retained to avoid requiring additional database privileges to create a separate schema. The snapshot serves as the historical tracking table, while DIM_INVENTORY remains the Gold Layer dimension consumed by downstream analytics.

Updating DIM_INVENTORY

The existing models/dimensions/dim_inventory.sql model was modified to read from inventory_snapshot instead of directly reading from STG_INVENTORY .

The model exposes the inventory attributes along with three historical-tracking columns:

Column	Description
VALID_FROM	Timestamp when the inventory version became valid
VALID_TO	Timestamp when the inventory version was superseded; NULL indicates the current version
IS_CURRENT	Boolean indicator identifying the current version

The model is materialized as a table and rebuilt from the snapshot history when executed. This allows the dimension to expose both current and historical inventory versions.

Since a product can have multiple historical versions, PRODUCT_ID alone is no longer sufficient to uniquely identify a row in DIM_INVENTORY . The logical version key is the combination of PRODUCT_ID and VALID_FROM .

Execution and Verification

The following commands were executed successfully:

```
dbt snapshot --select inventory_snapshot
dbt run --select dim_inventory
```

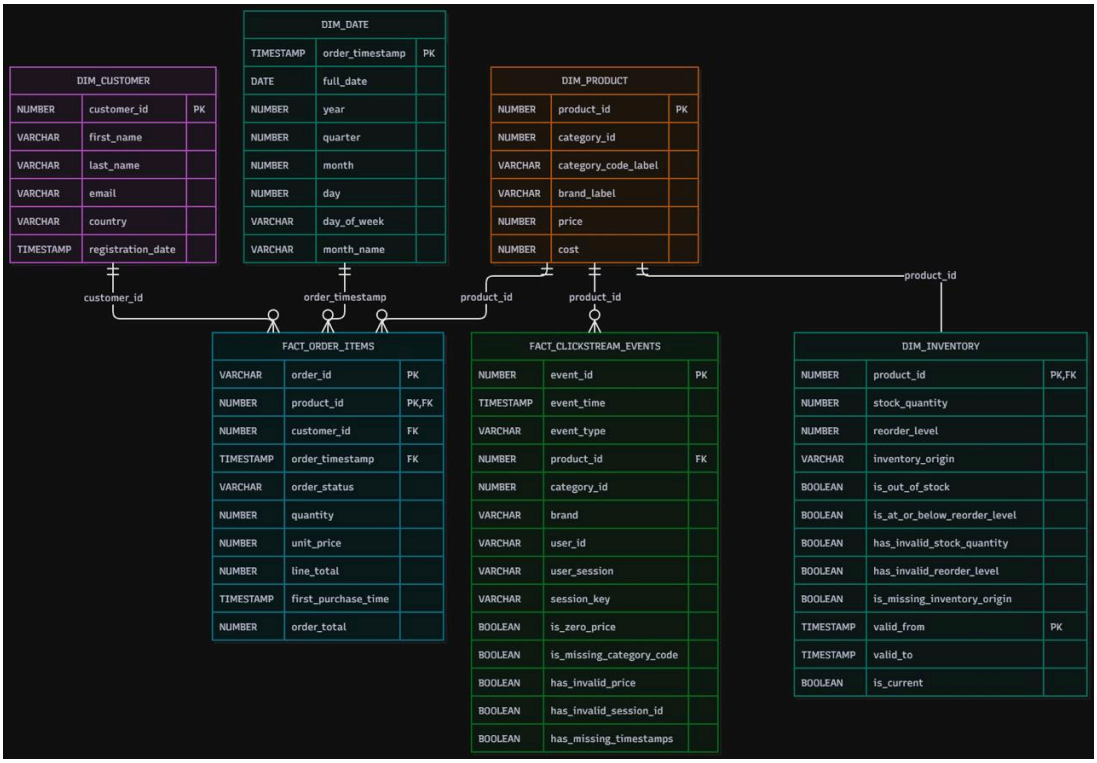
The initial snapshot processed 166,794 rows, and the updated DIM_INVENTORY table was created with 166,794 rows.

A read-only verification query returned the following results:

Validation	Result
Total rows	166,794
Unique products	166,794
Current versions	166,794
Records marked current	166,794
Records with null VALID_FROM	0

These results confirm that the initial snapshot and dimension structure were created successfully, with one current version per product at initialization.

Gold Layer Schema Design



Analytics, BI & Orchestration — Team Documentation Pending

Gold Analytical Serving Layer, Live BI Dashboard & Snowflake Native DAG Orchestration

1. Executive Summary & Architectural Plan

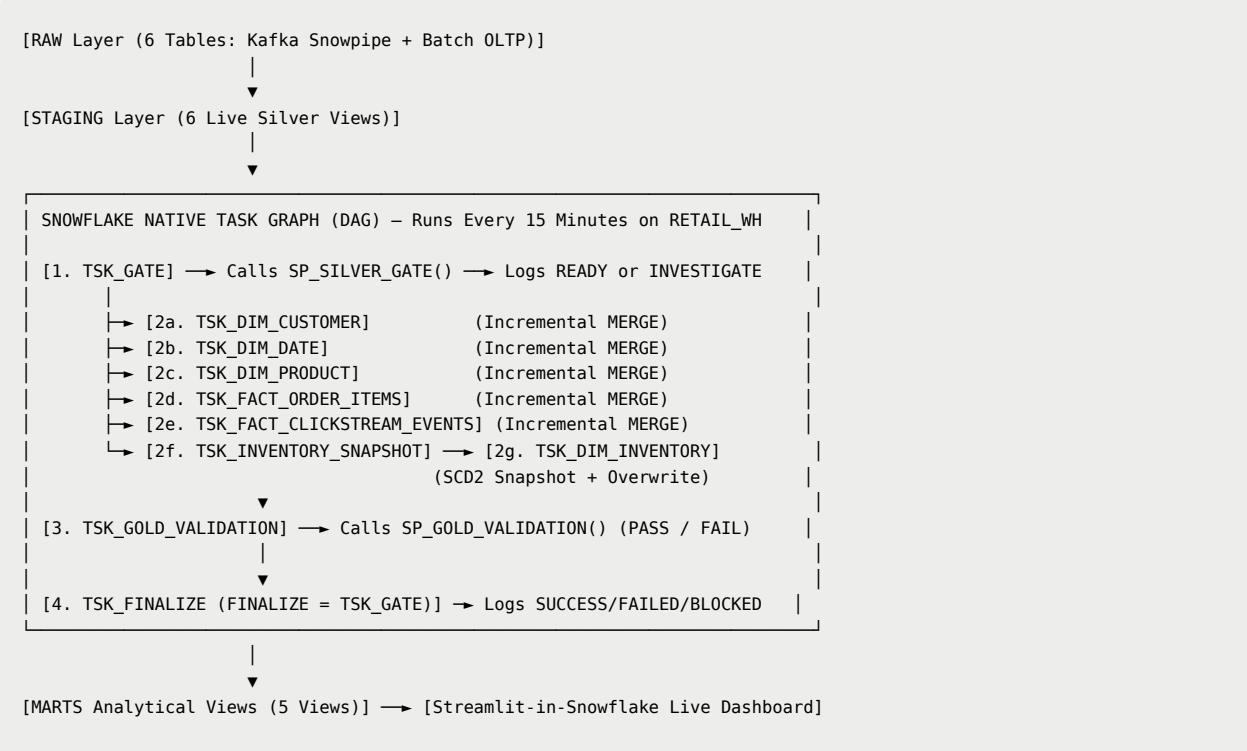
This module completes the end-to-end Retail Clickstream data platform by connecting the Silver (STAGING) and Gold (MARTS) layers to an automated Snowflake Native Task Graph (DAG) and a live Streamlit-in-Snowflake BI Dashboard.

1.1 Why This Design Was Chosen

1. **100% Cloud-Native Execution (Replacing Local dbt Core Runs):** The Gold tables in RETAIL_CLICKSTREAM_DB.MARTS were initially built using dbt Core on a local machine. Because cloud task schedulers cannot trigger a laptop-based dbt installation, all Gold incremental merges and the SCD Type 2 inventory snapshot were translated into idempotent, plain-SQL Snowflake Tasks inside RETAIL_CLICKSTREAM_DB.MARTS.
2. **Zero-Refresh Silver Layer:** The 6 Silver objects in RETAIL_CLICKSTREAM_DB.STAGING (STG_CLICKSTREAM_EVENTS, STG_CUSTOMERS, STG_ORDERS, STG_ORDER_ITEMS, STG_PRODUCTS, STG_INVENTORY) are materialized as live SQL views over the RAW tables, meaning they always reflect real-time Snowpipe and OLTP data without requiring separate staging refresh tasks.

3. **Pre-Flight & Post-Load Quality Gates:** Following the Silver-to-Gold handoff specification, the orchestration pipeline enforces a Pre-Load Silver Readiness Gate (SP_SILVER_GATE) before touching Gold tables, and a Post-Load Gold Validation Gate (SP_GOLD_VALIDATION) plus a Finalizer Task (TSK_FINALIZE) that logs every run's outcome into a central audit table (PIPELINE_RUN_LOG).

1.2 End-to-End Architecture Flow



2. Part 1: Analytical Serving Layer & Live Streamlit Dashboard

2.1 The Plan

Rather than forcing the dashboard to compute heavy joins across 150,000+ clickstream events, 690,000+ order items, and 3,000,000+ customers on every screen refresh, we designed a two-tier serving architecture inside RETAIL_CLICKSTREAM_DB.MARTS:

- **Tier 1 (5 SQL Analytical Views):** Pre-define session boundaries, funnel stages, dwell-time math, product profitability/ROI, and SCD Type 2 inventory health.
- **Tier 2 (Streamlit-in-Snowflake App):** Provides interactive filtering, dynamic data-quality and bot quarantine toggles, live auto-refresh, and explicit classification of Additive, Semi-Additive, and Non-Additive KPIs.

2.2 Step-by-Step Dashboard Implementation

Step 1: Access Control & Role Configuration (03_GRANT_PERMISSIONS.sql)

- Configured DBT_SILVER_ROLE and ANALYTICAL_USER on warehouse RETAIL_WH with privileges to read all RAW, STAGING, and MARTS objects and create views, tables, Streamlit apps, stages,

procedures, and tasks in RETAIL_CLICKSTREAM_DB.MARTS.

Step 2: Building the 5 Gold Analytical Views (Gold_layer/views/)

Created five live views in RETAIL_CLICKSTREAM_DB.MARTS:

View Name	Source Tables	Purpose & Business Logic
VW_SESSION_SUMMARY	FACT_CLICKSTREAM_EVENTS	Reconstructs user sessions by session_key. Computes session_dwell_seconds (MAX(event_time) - MIN(event_time)), stage flags (reached_view, reached_cart, reached_remove_from_cart, reached_purchase), rolls up the 5 clickstream quality flags into has_quality_issue, and flags suspected bots (is_suspected_bot: ≥ 25 events in ≤ 5 seconds or > 100 events).
VW_KPI_FUNNEL_CONVERSION	FACT_CLICKSTREAM_EVENTS, VW_SESSION_SUMMARY	Aggregates unique sessions and total events per funnel stage, split by has_quality_issue and is_suspected_bot for dynamic filtering.
VW_KPI_CATEGORY_ABANDONMENT	FACT_CLICKSTREAM_EVENTS, DIM_PRODUCT	Computes view sessions, cart sessions, purchase sessions, abandoned cart sessions (has_cart = 1 AND has_purchase = 0), and cart_abandonment_pct grouped by category_code_label and brand_label.
VW_KPI_REVENUE_AND_ROI	FACT_ORDER_ITEMS, DIM_PRODUCT, DIM_CUSTOMER, DIM_DATE	Computes gross_revenue (line_total), total_cogs (quantity * cost), net_profit (gross_revenue - total_cogs), and product_roi_pct across dates, product categories, brands, countries, and order statuses.
VW_INVENTORY_SCD2_HEALTH	DIM_INVENTORY, DIM_PRODUCT	Exposes stock levels, reorder alerts (is_out_of_stock, is_at_or_below_reorder_level), inventory quality flags (has_inventory_quality_issue), and SCD Type 2 history columns (valid_from, valid_to, is_current).

Step 3: App Environment & Sidebar Controls (environment.yml & streamlit_app.py)

- **Dependencies (environment.yml):** Configured the Snowflake conda environment with python=3.11.*, pandas=2.3.3, plotly=7.1.0, snowflake-snowpark-python, and streamlit.
- **Live Auto-Refresh:** Added a sidebar toggle (Live Auto-Refresh) and interval slider (10, 30, 60, 120, 300 seconds) plus a manual Refresh now button that clears the 60-second query cache (@st.cache_data(ttl=60)) and triggers st.rerun().

- **Interactive Anomaly & Bot Quarantine Filters:** Added two sidebar toggles (Exclude sessions with data-quality issues and Exclude suspected bot sessions) that dynamically inject has_quality_issue = FALSE and is_suspected_bot = FALSE into downstream SQL queries so evaluators can watch KPIs reconcile in real time.

Step 4: Executive KPI Banner & Metric Classification

Built the top KPI banner explicitly separating metrics into the three required data warehousing classifications:

- **Additive Metrics (Summable across all dimensions):**
  - Revenue: SUM(gross_revenue) from VW_KPI_REVENUE_AND_ROI.
  - Sessions: COUNT(*) from VW_SESSION_SUMMARY.
  - Orders: COUNT(DISTINCT order_id) from VW_KPI_REVENUE_AND_ROI.
- **Semi-Additive Metrics (Summable across products, never across time):**
  - Stock Units: SUM(stock_quantity) WHERE is_current = TRUE from VW_INVENTORY_SCD2_HEALTH (41.67M units across 166.8K active products).
- **Non-Additive Metrics (Ratios always recomputed from summed components):**
  - Abandonment %: SUM(abandoned_sessions) / SUM(cart_sessions) * 100.
  - Conversion %: SUM(purchase_sessions) / COUNT() 100.
  - ROI % & Margin %: SUM(net_profit) / SUM(total_cogs) * 100 and SUM(net_profit) / SUM(gross_revenue) * 100.

Step 5: Building the 3 Analytical Dashboard Tabs

1. Tab 1 — 🧭 Clickstream Journey (render_funnel, render_abandonment, render_dwell):

- **Funnel Conversion & Drop-Off:** Plots the monotonic progression (Viewed $\rightarrow$ Cart or later $\rightarrow$ Purchased) alongside a stage drop-off table and a data anomaly diagnostic caption noting purchase sessions that bypassed cart or view events in the raw stream.
- **Cart Abandonment by Category / Brand:** Horizontal bar chart with Top-N slider, minimum cart-session threshold filter (to hide statistically tiny groups), and an overall average reference line.
- **Session Dwell-Time Distribution:** Configurable histogram (bin width 5–60s, cap 300–3600s, and a checkbox to exclude single-event 0s sessions) plus summary cards for Average, Median, 90th Percentile dwell time, and single-event session counts.

2. Tab 2 — 💰 Revenue & ROI (render_revenue_trend, render_category_brand_roi, render_geography_and_status):

- **Revenue, Cost & Net Profit Over Time:** Grouped bar and line chart with Daily/Monthly toggle and multi-select order_status filter (CANCELLED, DELIVERED, PENDING, PROCESSING, REFUNDED, SHIPPED).
- **Profitability & ROI by Category / Brand:** Ranks top groups by Net Profit (), $GrossRevenue()$, ROI (%), or Gross Margin (%) with a side-by-side financial breakdown table.

- **Customer Geography & Order Fulfillment:** Top 15 countries by Gross Revenue bar chart and an Orders Share by Status donut chart.

3. Tab 3 — 📦 Inventory, Quality & DAG Monitor (render_inventory_scd2, render_quality_gates, render_dag_monitor):

- **Inventory Health & SCD Type 2 Explorer:** Displays 5 stock KPIs, a stacked bar chart of stock health across the 10 inventory origins (Warehouse, Store, Supplier, Dropship, etc.), and an interactive dropdown to inspect valid_from, valid_to, and is_current history per product.
- **Data Quality Gate Audit:** Audits all 9 quality and anomaly flags across Clickstream, Sessions, and Inventory (catching 48,387 missing category events, 161 zero-price events, and 4 suspected bot sessions).
- **Gold Freshness & Native Task DAG Monitor:** Live row counts and latest timestamps across all Gold tables alongside live outputs from SHOW TASKS and INFORMATION_SCHEMA.TASK_HISTORY filtered to RETAIL_CLICKSTREAM_DB.

3. Part 2: Snowflake Native DAG Orchestration

3.1 The Plan

To automate the Gold layer refreshes without relying on external dbt Core execution, we organized all orchestration code into numbered scripts inside 06_orchestration/ and built a 10-task Snowflake Native DAG in RETAIL_CLICKSTREAM_DB.MARTS scheduled every 15 minutes.

Task Name	Role in DAG	Predecessors (AFTER)	Execution Condition / Behavior
TSK_GATE	Root Task (15 MINUTE schedule)	None (Root)	Calls SP_SILVER_GATE(), logs metrics to PIPELINE_RUN_LOG, and publishes 'READY' or 'INVESTIGATE'.
TSK_DIM_CUSTOMER	Parallel Branch 1	TSK_GATE	Runs when gate = 'READY'. Incremental MERGE from STG_CUSTOMERS.
TSK_DIM_DATE	Parallel Branch 2	TSK_GATE	Runs when gate = 'READY'. Incremental MERGE of distinct timestamps from STG_ORDERS.
TSK_DIM_PRODUCT	Parallel Branch 3	TSK_GATE	Runs when gate = 'READY'. Incremental MERGE from STG_PRODUCTS.
TSK_FACT_ORDER_ITEMS	Parallel Branch 4	TSK_GATE	Runs when gate = 'READY'. Incremental MERGE from STG_ORDER_ITEMS ⋈ STG_ORDERS.
TSK_FACT_CLICKSTREAM_EVENTS	Parallel Branch 5	TSK_GATE	Runs when gate = 'READY'. Incremental MERGE from STG_CLICKSTREAM_EVENTS.

Task Name	Role in DAG	Predecessors (AFTER)	Execution Condition / Behavior
TSK_INVENTORY_SNAPSHOT	Parallel Branch 6 (Step A)	TSK_GATE	Runs when gate = 'READY'. Atomic SCD2 MERGE into INVENTORY_SNAPSHOT; publishes 'DONE'.
TSK_DIM_INVENTORY	Parallel Branch 6 (Step B)	TSK_INVENTORY_SNAPSHOT	Runs when snapshot = 'DONE'. Atomically rebuilds DIM_INVENTORY via INSERT OVERWRITE.
TSK_GOLD_VALIDATION	Fan-In Validation Task	All 6 Leaf Gold Tasks	Calls SP_GOLD_VALIDATION() to verify row counts, keys, orphans, and SCD2 integrity.
TSK_FINALIZE	Graph Finalizer (FINALIZE = TSK_GATE)	Attached to Root (TSK_GATE)	Always executes at graph completion; calls SP_FINALIZE_PIPELINE() to log SUCCESS, FAILED, or BLOCKED.

3.2 Step-by-Step Orchestration Implementation

Step 1: Plain-SQL Gold Incremental Merges (01_gold_refresh_statements.sql)

- Converted the 5 dbt incremental models (DIM_CUSTOMER, DIM_DATE, DIM_PRODUCT, FACT_ORDER_ITEMS, FACT_CLICKSTREAM_EVENTS) into standalone MERGE INTO statements.
- Added NULL-safe IS DISTINCT FROM guards on the WHEN MATCHED clauses so Snowflake only updates rows whose attributes actually changed, preventing unnecessary micro-partition rewrites and reporting exact inserted/updated row counts.

Step 2: Atomic SCD Type 2 Inventory Pipeline (02_inventory_scd2.sql)

- **Snapshot Refresh (INVENTORY_SNAPSHOT):** Replicated dbt's check snapshot strategy in a single atomic MERGE statement using a two-branch UNION ALL:
  - **Branch 1 (Close Old Version):** Emits changed products with their real PRODUCT_ID AS MERGE_KEY, matching the active snapshot row (DBT_VALID_TO IS NULL) and closing it by setting DBT_VALID_TO = RUN_TS.
  - **Branch 2 (Insert New Version):** Emits both new and changed products with NULL AS MERGE_KEY so they never match an existing row and are always inserted as the new current version (DBT_VALID_FROM = RUN_TS, DBT_VALID_TO = NULL, and an MD5 DBT_SCD_ID).
- **Dimension Rebuild (DIM_INVENTORY):** Used INSERT OVERWRITE INTO RETAIL_CLICKSTREAM_DB.MARTS.DIM_INVENTORY (instead of CREATE OR REPLACE TABLE) so the table is refreshed atomically while preserving all role privileges and dependent dashboard views.

Step 3: Audit Logging & Pre-Flight Silver Readiness Gate (03_log_and_gate.sql)

- **Audit Table (PIPELINE_RUN_LOG):** Created RETAIL_CLICKSTREAM_DB.MARTS.PIPELINE_RUN_LOG storing LOG_ID, RUN_ID (capturing SYSTEM$TASK_RUNTIME_INFO('CURRENT_TASK_GRAPH_RUN_GROUP_ID') so every task in a single DAG execution shares one ID), LOGGED_AT, STEP_NAME, STATUS, MESSAGE, and a JSON METRICS variant.
- **Gate Procedure (SP_SILVER_GATE):** Extended 01_silver_readiness.sql to check in a single consistent snapshot:
  1. RAW_CLICKSTREAM_EVENTS vs. STG_CLICKSTREAM_EVENTS row-count equality.
  2. Zero NULL or duplicate EVENT_ID rows in STG_CLICKSTREAM_EVENTS.
  3. Zero NULL or duplicate business keys across STG_CUSTOMERS, STG_PRODUCTS, STG_INVENTORY, and STG_ORDER_ITEMS ⋈ STG_ORDERS (protecting downstream MERGE statements from duplicate-key aborts or NULL-key bloat).
- If all checks pass, SP_SILVER_GATE logs READY and publishes 'READY' via SYSTEM$SET_RETURN_VALUE; otherwise, it logs INVESTIGATE with the exact failure diagnostics and blocks all downstream Gold tasks.

Step 4: Creating the Core Task Graph (04_create_tasks.sql)

- Created TSK_GATE and the 7 Gold refresh tasks in a suspended state with production safety guardrails: ALLOW_OVERLAPPING_EXECUTION = FALSE, SUSPEND_TASK_AFTER_NUM_FAILURES = 3, and USER_TASK_TIMEOUT_MS = 600000 (10 minutes).

Step 5: Post-Load Gold Validation & Finalizer (05_validation_and_finalizer.sql & 07_fix_validation_task.sql)

- **SP_GOLD_VALIDATION & TSK_GOLD_VALIDATION:** Wired TSK_GOLD_VALIDATION to run AFTER all 6 leaf Gold tasks (TSK_DIM_CUSTOMER, TSK_DIM_DATE, TSK_DIM_PRODUCT, TSK_FACT_ORDER_ITEMS, TSK_FACT_CLICKSTREAM_EVENTS, TSK_DIM_INVENTORY). It verifies that Gold row counts match Staging distinct-key counts, checks for zero NULL/duplicate Gold keys, confirms zero orphan PRODUCT_IDs in Facts vs. DIM_PRODUCT, and verifies that no product has multiple IS_CURRENT = TRUE rows in DIM_INVENTORY. If any check fails, it logs FAIL to PIPELINE_RUN_LOG and raises exception -20001 so the task fails visibly.
- **SP_FINALIZE_PIPELINE & TSK_FINALIZE:** Attached TSK_FINALIZE to the root task using FINALIZE = RETAIL_CLICKSTREAM_DB.MARTS.TSK_GATE. At the end of every graph run, it inspects PIPELINE_RUN_LOG and INFORMATION_SCHEMA.TASK_HISTORY for the current RUN_ID and logs a final STEP_NAME = 'PIPELINE' summary row (SUCCESS, FAILED, or BLOCKED).

4. Part 3: End-to-End Verification & Technical Evidence (06 & 08 Scripts)

All four operational paths were tested and verified in Snowflake (06_test_and_scd2_proof.sql, 07_fix_validation_task.sql, and 08_failure_paths_and_scd2_proof.sql):

1. Clean End-to-End Run (SUCCESS Path):

- Enabled all child tasks via SYSTEM$TASK_DEPENDENTS_ENABLE('RETAIL_CLICKSTREAM_DB.MARTS.TSK_GATE') and triggered EXECUTE TASK TSK_GATE.
- All 10 tasks in the graph completed with STATE = 'SUCCEEDED', and PIPELINE_RUN_LOG recorded 3 rows sharing the same RUN_ID: SILVER_GATE = READY → GOLD_VALIDATION = PASS → PIPELINE = SUCCESS.

2. Gate Failure Test (BLOCKED Path):

- Temporarily modified TSK_GATE to call SP_SILVER_GATE(TRUE) (forced failure mode) and executed the graph.
- Verified that TSK_GATE returned INVESTIGATE, all Gold refresh tasks were SKIPPED (protecting Gold tables from corrupt upstream data), and TSK_FINALIZE logged PIPELINE = BLOCKED before restoring TSK_GATE.

3. Downstream Failure Test (FAILED Path):

- Temporarily modified TSK_GOLD_VALIDATION to call SP_GOLD_VALIDATION(TRUE) and executed the graph.
- Verified that TSK_GOLD_VALIDATION failed with error -20001, logged GOLD_VALIDATION = FAIL, and TSK_FINALIZE logged PIPELINE = FAILED naming TSK_GOLD_VALIDATION as the failed task before restoring the original task definition.
- *4. Non-Destructive SCD Type 2 Proof (ZZ_SCD2_TEST_*):**
- Cloned INVENTORY_SNAPSHOT and copied STG_INVENTORY into transient scratch tables (ZZ_SCD2_TEST_SNAPSHOT and ZZ_SCD2_TEST_STG) so production tables remained untouched.
- Incremented STOCK_QUANTITY by +100 for one product (MIN(PRODUCT_ID)) and ran the atomic snapshot MERGE.
- Verified that the product's previous version was closed (DBT_VALID_TO populated, IS_CURRENT = FALSE) and a new current version was inserted (DBT_VALID_FROM = old VALID_TO, DBT_VALID_TO = NULL, IS_CURRENT = TRUE), increasing total rows from 166,794 to 166,795 while keeping current_rows = 166,794 and products_with_multiple_current = 0.
- Re-ran the exact MERGE a second time and confirmed 0 rows inserted, 0 rows updated (proving idempotency), then dropped both scratch tables.

5. Part 4: Team Operational Runbook & Governance Rules

5.1 Important Rule for the Team

- **Do NOT run dbt run or dbt snapshot against the MARTS schema while the Snowflake Task Graph is active.** The Snowflake Native DAG now owns the lifecycle of all 6 Gold tables and INVENTORY_SNAPSHOT. Running dbt manually from a laptop could cause concurrent table locks or schema/snapshot conflicts.

5.2 Essential Commands for Demo & Credit Management

Trigger an Immediate Manual Run (During Live Presentation):

```
USE ROLE DBT_SILVER_ROLE;
EXECUTE TASK RETAIL_CLICKSTREAM_DB.MARTS.TSK_GATE;
```

Inspect the Latest Pipeline Audit Logs:

```
SELECT LOG_ID, RUN_ID, LOGGED_AT, STEP_NAME, STATUS, MESSAGE, METRICS
FROM RETAIL_CLICKSTREAM_DB.MARTS.PIPELINE_RUN_LOG
ORDER BY LOGGED_AT DESC
LIMIT 15;
```

Pause the Schedule After Demo / Testing (To Save Snowflake Credits):

```
ALTER TASK RETAIL_CLICKSTREAM_DB.MARTS.TSK_GATE SUSPEND;
```

Resume the 15-Minute Schedule Before the Presentation:

```
SELECT SYSTEM$TASK_DEPENDENTS_ENABLE('RETAIL_CLICKSTREAM_DB.MARTS.TSK_GATE');
ALTER TASK RETAIL_CLICKSTREAM_DB.MARTS.TSK_GATE RESUME;
```

6. Repository & Workspace File Inventory

Folder / Location	File Name	Description
Gold_layer/views/	vw_session_summary.sql	Session reconstruction, dwell time, funnel flags, quality & bot flags.
Gold_layer/views/	vw_kpi_funnel_conversion.sql	Stage-by-stage session and event counts for funnel visualization.
Gold_layer/views/	vw_kpi_category_abandonment.sql	Cart abandonment rate by category and brand.
Gold_layer/views/	vw_kpi_revenue_and_roi.sql	Gross revenue, COGS, net profit, and ROI across dimensions.
Gold_layer/views/	vw_inventory_scd2_health.sql	Current stock health, reorder alerts, and SCD Type 2 version history.
Streamlit App	streamlit_app.py & environment.yml	3-tab interactive Streamlit-in-Snowflake dashboard with live refresh.
06_orchestration/	01_gold_refresh_statements.sql	Plain-SQL incremental MERGE statements for the 5 Gold tables.
06_orchestration/	02_inventory_scd2.sql	Atomic INVENTORY_SNAPSHOT SCD2 MERGE and DIM_INVENTORY overwrite.
06_orchestration/	03_log_and_gate.sql	Creates PIPELINE_RUN_LOG and SP_SILVER_GATE readiness procedure.
06_orchestration/	04_create_tasks.sql	Creates TSK_GATE and the 7 Gold refresh tasks.
06_orchestration/	05_validation_and_finalizer.sql	Creates SP_GOLD_VALIDATION, SP_FINALIZE_PIPELINE, TSK_GOLD_VALIDATION, and TSK_FINALIZE.

Folder / Location	File Name	Description
06_orchestration/	06_test_and_scd2_proof.sql & 07_fix_validation_task.sql	Enables task dependents and validates clean end-to-end DAG execution.
06_orchestration/	08_failure_paths_and_scd2_proof.sql	Proves BLOCKED path, FAILED path, and non-destructive SCD Type 2 versioning.